

Predictive Modeling for Financial Risk Assessment

Loan Default Prediction Using a Random Forest Classifier

by

Katherine Rivera

A Graduate Capstone Report Completed in Fulfillment of the Requirements for the Degree of
Master of Science in Data Analytics – Data Science

School of Technology
Western Governors University

August 2026

Table of Contents

Executive Summary	3
Research Question	4
Data Collection	5
Data Extraction and Preparation	6
Analysis	20
Data Summary and Implications	44
References	49

Executive Summary

The purpose of this study is to encompass the steps and processes of the data analysis lifecycle by identifying borrower, loan, and credit factors associated with loan default, and to further explore whether a Random Forest classification model can predict the risk of loan default. The dataset provided by Western Governors University comprises 148,670 loan records. The analysis encompassed the steps of the data analytics lifecycle, such as understanding the organizational or business needs to formulate a research question, data collection methods, data extraction and preparation, analysis of the data (e.g., exploratory data analysis, inferential statistical testing, model development and training, hyperparameter tuning, deployment), and thorough discussion of the results, implications, limitations and possibility for future studies. The results demonstrated statistically significant relationships between loan default status and most of the selected variables. The optimized Random Forest achieved moderately strong metric scores for accuracy, F1 score, and ROC-AUC, and improved performance in minimizing the number of actual missed loan defaults. This final model was then deployed as a FastAPI prototype to showcase predictions on whether a new loan application is high- or low-risk of default. Overall, the findings of this study demonstrate sufficient evidence that the use of predictive modeling can support data-driven decision-making as a support tool, while considering that expert judgment, implementation of fairness and bias measures, and continued monitoring and security measures are to be addressed before the model is set to be used in real-life operational situations.

Research Question

The purpose of this analysis is to explore and provide answers and next steps for the following research question: Which characteristics of borrowers, loans, and credit are statistically associated with loan default status, and can an optimized Random Forest classification model accurately predict the risk of defaulting on a loan? This is an important question to consider, as the ability to assess loan default risk is critical for many banks and financial companies. Missed opportunities for financial gains may be at stake, and decisions to approve loans to borrowers who may eventually default could result in revenue loss. Standard approval rules may not fully capture the complex, potentially underlying relationships among borrower characteristics, loan terms, credit factors, and application details.

Financial companies and bank stakeholders often want to ensure they make the best decisions when choosing qualified borrowers for loans. They want to capture revenue opportunities, maintain positive relationships with their consumers, and ultimately keep the risk of revenue loss low. Therefore, financial companies may be tempted to investigate which characteristics of potential borrowers make them more likely to default on a loan if granted one. An inaccurate default-risk assessment can lead to financial loss. Banks may be losing out on money owed by borrowers, therefore needing to increase fees for those borrowers and limiting loan opportunities for banks to lend to future borrowers. However, standard approval rules may not fully capture the complex relationships among borrower characteristics, loan terms, credit factors, and application details; these features of the loans themselves and of credit status may contribute to increased loan default risk. Stakeholders in financial companies and banks may seek to understand possible associations between these factors and the likelihood of loan default, and to make accurate predictions, so they can make strategic decisions about whether to accept loan applications. The performance of these predictions will be evaluated utilizing metrics such as accuracy, precision, recall, F1-score, ROC-AUC, and the confusion matrix. Furthermore, with predictive modeling, this type of analysis can support lending risk assessment by identifying which factors within borrowers and their credit are more likely to flag borrowers as having a higher risk of defaulting on their loans if approved, and to what degree. Lastly, statistical testing is useful for identifying significant relationships among borrowers, other related factors, and the target variable, Status, so that, if there is indeed an underlying relationship, it can be examined further to determine what kinds of relationships exist. If not, then it can be a good indication of additional data and factors that may need to be taken into consideration to best address the stakeholders' needs.

To address the proposed research question, the null hypothesis to be tested in this analysis is as follows: There is no statistically significant relationship between borrower, loan, and credit-related factors and loan default status. The alternative hypothesis, then, in this scenario, is the following: There is a statistically significant relationship between one or more borrower, loan, and credit-related factors and loan default status. The hypothesis aligns with the research question in this analysis, in that it is first needed to examine whether there is a significant

relationship between such factors and loan default status, and if so, whether a Random Forest classification model can accurately predict the risk of defaulting on a loan based on identified characteristics after optimization has been applied. These hypotheses will be tested using inferential statistics to determine whether selected borrower-, loan-, and credit-related factors have statistically significant relationships with loan default status. A significance level of 0.05 will be utilized as the alpha threshold for this analysis. The null and alternative hypotheses focus on the inferential statistical testing component of the research question, while the performance of the Random Forest model will be evaluated separately using classification metrics.

Data Collection

The secondary data used in this analysis were obtained from the WGU loan default dataset, in Excel format, available in the Capstone Course (Western Governors University, 2026). Additional data from other sources have not been, nor will they be, gathered for this analysis due to proprietary concerns. The available dataset contains 34 variables and 148,670 records. The variables represent various features, including loan amount, interest rate, creditworthiness, income, credit score, age, etc. The dataset also contains a mixture of both quantitative and qualitative variables. For this analysis, the Status variable is identified as the target variable because it indicates whether a loan has defaulted.

Having a readily accessible Excel format for the dataset can allow for easier access to the data needed for analysis. The dataset is presented in a structured, tabular format that allows a more efficient import into Python and wrangling, although much-needed data inspection and preprocessing were still essential prior to any statistical testing or modeling. One disadvantage, however, of using an existing provided dataset is that the data collection process is limited for the analyst. Hence, the analysis may be limited to variables already included in the dataset, and some crucial lending factors may be unavailable for analysis. Furthermore, the dataset currently contains missing values across variables, raising questions about why these variables are missing. Additional documentation on the reasons behind these missing values was unavailable. Since I did not have much control over the data-collection process, I was unable to recover the true missing values. However, to address this challenge, I quantified the amount of missing values for each variable through the inspection component of the analysis, kept the variables that would be relevant to the research question and analysis, and applied appropriate preprocessing steps, such as handling missing values, encoding categorical variables, evaluating outliers, etc., prior to conducting inferential statistical tests and modeling.

Data Extraction and Preparation

The dataset file was downloaded from the WGU-provided course link and was saved in the project folder created for this analysis. The dataset was then loaded into a Pandas DataFrame using Python. The first 10 rows were also quickly inspected:

```
#Importing dataset and inspecting first few rows
df = pd.read_excel('Loan_Default_Data.xlsx')
df.head(10)
```

	ID	year	loan_limit	Gender	approv_in_adv	loan_type	loan_purpose	Credit_Worthiness	open_credit	business_or_commercial	...	credit_type	Credit_Score	co-applicant_credit_type
0	24890	2019	cf	Sex Not Available	nopre	type1	p1	I1	nopc	nob/c	...	EXP	758	CIB
1	24891	2019	cf	Male	nopre	type2	p1	I1	nopc	b/c	...	EQUI	552	EXP
2	24892	2019	cf	Male	pre	type1	p1	I1	nopc	nob/c	...	EXP	834	CIB
3	24893	2019	cf	Male	nopre	type1	p4	I1	nopc	nob/c	...	EXP	587	CIB
4	24894	2019	cf	Joint	pre	type1	p1	I1	nopc	nob/c	...	CRIF	602	EXP
5	24895	2019	cf	Joint	pre	type1	p1	I1	nopc	nob/c	...	EXP	864	EXP
6	24896	2019	cf	Joint	pre	type1	p3	I1	nopc	nob/c	...	EXP	860	EXP
7	24897	2019	NaN	Female	nopre	type1	p4	I1	nopc	nob/c	...	CIB	863	CIB
8	24898	2019	cf	Joint	nopre	type1	p3	I1	nopc	nob/c	...	CIB	580	EXP
9	24899	2019	cf	Sex Not Available	nopre	type3	p3	I1	nopc	nob/c	...	CIB	788	EXP

10 rows x 34 columns

The process now involves inspecting the dataset for data quality issues that must be addressed through cleaning and preprocessing the data prior to modeling. To start, the structure of the dataset was further inspected, including the variables, the amount of non-null values, the shape of the dataset, and the value counts for the target variable, identified as the Status variable:

```
#Further inspection of dataset
df.info()

<class 'pandas.DataFrame'>
RangeIndex: 148670 entries, 0 to 148669
Data columns (total 34 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   ID                                     148670 non-null  int64
1   year                                 148670 non-null  int64
2   loan_limit                           145326 non-null  str
3   Gender                               148670 non-null  str
4   approv_in_adv                        147762 non-null  str
5   loan_type                            148670 non-null  str
6   loan_purpose                           148536 non-null  str
7   Credit_Worthiness                   148670 non-null  str
8   open_credit                         148670 non-null  str
9   business_or_commercial               148670 non-null  str
10  loan_amount                          148670 non-null  int64
11  rate_of_interest                     112231 non-null  float64
12  Interest_rate_spread                 112031 non-null  float64
13  Upfront_charges                      109028 non-null  float64
14  term                                 148629 non-null  float64
15  Neg_ammortization                    148549 non-null  str
16  interest_only                        148670 non-null  str
17  lump_sum_payment                     148670 non-null  str
18  property_value                       133572 non-null  float64
19  construction_type                   148670 non-null  str
20  occupancy_type                       148670 non-null  str
21  Secured_by                           148670 non-null  str
22  total_units                          148670 non-null  str
23  income                               139520 non-null  float64
24  credit_type                          148670 non-null  str
25  Credit_Score                         148670 non-null  int64
26  co-applicant_credit_type             148670 non-null  str
27  age                                  148470 non-null  str
28  submission_of_application            148470 non-null  str
29  LTV                                  133572 non-null  float64
30  Region                               148670 non-null  str
31  Security_Type                       148670 non-null  str
32  Status                               148670 non-null  int64
33  dtirl                                124549 non-null  float64
dtypes: float64(8), int64(5), str(21)
memory usage: 38.6 MB
```

```
#Further inspection of dataset cont..
print(df.shape)
print(df.columns)
print(df['Status'].value_counts())

(148670, 34)
Index(['ID', 'year', 'loan_limit', 'Gender', 'approv_in_adv', 'loan_type',
       'loan_purpose', 'Credit_Worthiness', 'open_credit',
       'business_or_commercial', 'loan_amount', 'rate_of_interest',
       'Interest_rate_spread', 'Upfront_charges', 'term', 'Neg_ammortization',
       'interest_only', 'lump_sum_payment', 'property_value',
       'construction_type', 'occupancy_type', 'Secured_by', 'total_units',
       'income', 'credit_type', 'Credit_Score', 'co-applicant_credit_type',
       'age', 'submission_of_application', 'LTV', 'Region', 'Security_Type',
       'Status', 'dtirl'],
      dtype='str')
Status
0    112031
1     36639
Name: count, dtype: int64
```

The dataset has a shape of 148,670 rows and 34 columns. Furthermore, based on the non-null values, the dataset contains missing values from various variables. The status variable takes values 0 and 1, with 0 indicating the borrower did not default on a loan and 1 indicating the borrower did. The status variable has a total “0” count of 112,031 and a total “1” count of 36,639. This indicates an imbalance in the status variable.

Checking for missing values is an important next step in determining appropriate strategies for handling them in the analysis. Several variables contained missing values, and are sorted in descending order:

```
#Identifying missing values
print(df.isnull().sum().sort_values(ascending = False))
```

Upfront_charges	39642
Interest_rate_spread	36639
rate_of_interest	36439
dtir1	24121
property_value	15098
LTV	15098
income	9150
loan_limit	3344
approv_in_adv	908
age	200
submission_of_application	200
loan_purpose	134
Neg_ammortization	121
term	41
year	0
ID	0
business_or_commercial	0
open_credit	0

Inspecting for any duplicate records was also done, and the dataset was found to contain no duplicate records or rows:

```
#Identifying any duplicate rows
duplicates = df[df.duplicated()]
duplicates
```

ID	year	loan_limit	Gender	approv_in_adv	loan_type	loan_purpose	Credit_Worthiness
----	------	------------	--------	---------------	-----------	--------------	-------------------

0 rows x 34 columns

The data type for each of the variables in the dataset was also inspected to ensure that the variables had the correct data type:

```
#Checking data types
df.dtypes
```

ID	int64
year	int64
loan_limit	str
Gender	str
approv_in_adv	str
loan_type	str
loan_purpose	str
Credit_Worthiness	str
open_credit	str
business_or_commercial	str
loan_amount	int64
rate_of_interest	float64
Interest_rate_spread	float64
Upfront_charges	float64
term	float64
Neg_ammortization	str
interest_only	str
lump_sum_payment	str
property_value	float64
construction_type	str
occupancy_type	str
Secured_by	str
total_units	str
income	float64
credit_type	str
Credit_Score	int64
co-applicant_credit_type	str
age	str
submission_of_application	str
LTV	float64
Region	str
Security_Type	str
Status	int64
dtir1	float64
dtype:	object

The variable Age, one of the variables to be selected for the analysis, is labeled as a string or object variable. Because the age variable contains entries such as “25-34”, “35-44”, etc., it needs to be converted into a category during the cleaning phase to ensure the age ranges are labeled accordingly.

The value counts for each categorical column in the dataset were further examined to identify inconsistencies or blank labels. Within the Security_Type variable, the entry “Indriect” is misspelled. However, because this variable was not selected for the inferential or predictive modeling analysis, the inconsistency therefore did not affect the analysis, and no correction was required.

```
#Identifying categorical columns in dataset and reviewing value counts
obj_col = df.select_dtypes(include = ['object', 'string', 'category']).columns

for col in obj_col:
    print('Value counts for:', df[col].value_counts(dropna = False))

Value counts for: loan_limit
cf      135348
ncf      9978
NaN      3344
Name: count, dtype: int64
Value counts for: Gender
Male      42346
Joint      41399
Sex Not Available  37659
Female      27266
Name: count, dtype: int64
Value counts for: approv_in_adv
nopre     124621
pre        23141
NaN         908
Name: count, dtype: int64
Value counts for: loan_type
type1     113173
type2      20762
type3      14735
Name: count, dtype: int64
Value counts for: loan_purpose
p3         55934
p4         54799
p1         34529
p2          3274
NaN         134
Name: count, dtype: int64
Value counts for: Credit_Worthiness
11         142344
12          6326
Name: count, dtype: int64
Value counts for: open_credit
nopc       148114
opc          556
Name: count, dtype: int64
Value counts for: business_or_commercial
nob/c      127908
b/c        20762

Value counts for: Security_Type
direct     148637
Indriect       33
Name: count, dtype: int64
```

To prepare a Random Forest classification model to address the business need for this analysis, relevant variables were selected and assigned as a new dataframe labeled “df_keep”. These variables were identified as variables that can have a meaningful impact and associations with the target variable, and align with the overarching business problem in exploring whether factors relating to borrower, credit, and application are statistically significant to loan default status, and whether these factors can accurately predict whether a borrower will default on a loan. The ID variable was excluded because it served only as an identifier. The other excluded variables did

not align directly with the defined research question. These 20 variables, separate from the target variable, were cleaned and preprocessed, and further investigated using inferential statistical tests to determine the statistical significance of each variable with respect to the target variable, Status.

```
#Keeping necessary columns for analysis
keep_columns = [
    'credit_type',
    'loan_type',
    'loan_purpose',
    'loan_limit',
    'approv_in_adv',
    'Credit_Worthiness',
    'business_or_commercial',
    'loan_amount',
    'term',
    'Neg_ammortization',
    'interest_only',
    'lump_sum_payment',
    'income',
    'Credit_Score',
    'age',
    'submission_of_application',
    'LTV',
    'dtir1',
    'occupancy_type',
    'Region']

#New dataframe with kept variables and target variable
df_keep = df[keep_columns + ['Status']]
df_keep.head(10)
```

	credit_type	loan_type	loan_purpose	loan_limit	approv_in_adv	Credit_Worthiness	business_or_commercial	loan_amount	term	Neg_ammortization	...	lump_sum_payment	income	Credit_Score	age	submission_of_application
0	EXP	type1	p1	cf	no pre	11	no b/c	116500	360.0	not_neg	..	not_lpsm	1740.0	758	25-34	to_int
1	EQUI	type2	p1	cf	no pre	11	b/c	206500	360.0	not_neg	..	lpsm	4980.0	552	55-64	to_int
2	EXP	type1	p1	cf	pre	11	no b/c	406500	360.0	neg_amm	..	not_lpsm	9480.0	834	35-44	to_int
3	EXP	type1	p4	cf	no pre	11	no b/c	456500	360.0	not_neg	..	not_lpsm	11880.0	587	45-54	not_int
4	CRIF	type1	p1	cf	pre	11	no b/c	696500	360.0	not_neg	..	not_lpsm	10440.0	602	25-34	not_int
5	EXP	type1	p1	cf	pre	11	no b/c	706500	360.0	not_neg	..	not_lpsm	10080.0	864	35-44	not_int
6	EXP	type1	p3	cf	pre	11	no b/c	346500	360.0	not_neg	..	not_lpsm	5040.0	860	55-64	to_int
7	CIB	type1	p4	NaN	no pre	11	no b/c	266500	360.0	not_neg	..	not_lpsm	3780.0	863	55-64	to_int
8	CIB	type1	p3	cf	no pre	11	no b/c	376500	360.0	not_neg	..	not_lpsm	5580.0	580	55-64	to_int
9	CIB	type3	p3	cf	no pre	11	no b/c	436500	360.0	not_neg	..	not_lpsm	6720.0	788	55-64	to_int

10 rows x 21 columns

The new dataframe is reinspected to verify that the correct variables have been selected:

```
#Verifying dataset of selected variables
df_keep.info()

<class 'pandas.DataFrame'>
RangeIndex: 148670 entries, 0 to 148669
Data columns (total 21 columns):
 #   Column                                Non-Null Count  Dtype
---  -
 0   credit_type                          148670 non-null  str
 1   loan_type                            148670 non-null  str
 2   loan_purpose                          148536 non-null  str
 3   loan_limit                          145326 non-null  str
 4   approv_in_adv                      147762 non-null  str
 5   Credit_Worthiness                   148670 non-null  str
 6   business_or_commercial              148670 non-null  str
 7   loan_amount                        148670 non-null  int64
 8   term                               148629 non-null  float64
 9   Neg_ammortization                   148549 non-null  str
10   interest_only                      148670 non-null  str
11   lump_sum_payment                   148670 non-null  str
12   income                             139520 non-null  float64
13   Credit_Score                       148670 non-null  int64
14   age                                148470 non-null  str
15   submission_of_application           148470 non-null  str
16   LTV                                133572 non-null  float64
17   dtir1                              124549 non-null  float64
18   occupancy_type                     148670 non-null  str
19   Region                             148670 non-null  str
20   Status                             148670 non-null  int64
dtypes: float64(4), int64(3), str(14)
memory usage: 23.8 MB

#Confirming shape
df_keep.shape

(148670, 21)
```

Within the selected variables, inspection for negative values was performed to determine whether any values fell outside the logical value ranges for the related lending measures. No negative values were found:

```
#Inspecting any rows in numeric columns that have negative values
num_cols = df_keep.select_dtypes(include = ['int64', 'float64']).columns.drop('Status')
neg_val = df_keep[(df_keep[num_cols] < 0).any(axis = 1)]
neg_val
```

credit_type	loan_type	loan_purpose	loan_limit	approv_in_adv	Credit_Worthiness	business_or_commercial	loan_amount	term	Neg_ammortization	...	lump_sum_payment	income	Credit_Score	age	submission_of_application
0 rows x 16 columns															

Possible outliers were also inspected using the IQR method:

```
#Inspecting for any outliers in the dataframe using the IQR method
outliers = pd.DataFrame(index = df_keep.index)

for col in num_cols:
    q1 = df_keep[col].quantile(0.25)
    q3 = df_keep[col].quantile(0.75)
    iqr = q3 - q1
    lower = q1 - 1.5 * iqr
    upper = q3 + 1.5 * iqr

    outliers[col] = (df_keep[col] < lower) | (df_keep[col] > upper)

outliers_rows = df_keep[outliers.any(axis = 1)]
outliers_rows.head(15)
```

	credit_type	loan_type	loan_purpose	loan_limit	approv_in_adv	Credit_Worthiness	business_or_commercial	loan_amount	term	Neg_ammortization	...	lump_sum_payment	income	Credit_Score
10	EXP	type2	p3	cf	no pre	I2	b/c	136500	300.0	neg_amm	...	not_lpsm	4020.0	723
20	EXP	type1	p3	cf	no pre	I1	nob/c	306500	180.0	not_neg	...	not_lpsm	16860.0	668
38	EXP	type1	p4	ncf	pre	I1	nob/c	256500	180.0	not_neg	...	not_lpsm	3780.0	780
44	CRIF	type1	p4	ncf	no pre	I1	nob/c	866500	360.0	neg_amm	...	not_lpsm	13620.0	876
50	CIB	type1	p3	cf	no pre	I1	nob/c	196500	312.0	not_neg	...	not_lpsm	9060.0	543
51	EXP	type1	p4	cf	no pre	I1	nob/c	206500	180.0	not_neg	...	not_lpsm	9600.0	846

Furthermore, the specific variables that contained outliers and the number of outliers were identified:

```
#Identifying specific variables that contain outliers
outliers_col = outliers.sum().sort_values(ascending = False)
outliers_col[outliers_col > 0]
```

```
term          26944
income        6546
dtir1         2013
loan_amount   1895
LTV           1882
dtype: int64
```

The outliers in these identified numeric variables may be reasonably explained. Some borrowers' incomes, term lengths, debt-to-income ratios, amounts borrowed, and loan-to-value ratios may be extremely high or low in some situations and may reflect actual conditions of the loans. Therefore, the decision to keep the outliers aligns with this realistic reasoning, as they may represent logical lending conditions. However, their effect should be considered when

interpreting the analysis, as extreme values can influence the summary statistics of the variables and the results of inferential statistical tests.

The summary statistics were then explored for the selected variables. The numeric variables were selected first and labeled as a new variable named `quan_cols`, in which the statistics were then examined:

```
#Summary statistics for Numeric columns
quan_cols = df_keep[['loan_amount',
                    'term',
                    'income',
                    'Credit_Score',
                    'LTV',
                    'dtir1']]

quan_stats = quan_cols.describe()
quan_stats.loc['range'] = quan_stats.loc['max'] - quan_stats.loc['min']
quan_stats.loc['variance'] = quan_cols.var()
quan_stats = quan_stats.round(2)

quan_stats
```

	loan_amount	term	income	Credit_Score	LTV	dtir1
count	1.486700e+05	148629.00	139520.00	148670.00	133572.00	124549.00
mean	3.311177e+05	335.14	6957.34	699.79	72.75	37.73
std	1.839093e+05	58.41	6496.59	115.88	39.97	10.55
min	1.650000e+04	96.00	0.00	500.00	0.97	5.00
25%	1.965000e+05	360.00	3720.00	599.00	60.47	31.00
50%	2.965000e+05	360.00	5760.00	699.00	75.14	39.00
75%	4.365000e+05	360.00	8520.00	800.00	86.18	45.00
max	3.576500e+06	360.00	578580.00	900.00	7831.25	61.00
range	3.560000e+06	264.00	578580.00	400.00	7830.28	56.00
variance	3.382263e+10	3411.62	42205634.62	13427.21	1597.41	111.21

The value counts, means, standard deviations, minimums, maximums, 25th, 50th, and 75th percentiles, ranges, and variances are displayed. Examining the statistics is another method for finding anomalies or inconsistencies within the data. According to the statistics, the maximum credit score is 900. Although base FICO scores range from 300 to 850, some industry-specific FICO models can reach 900 (FICO score types, n.d.). Since there were credit score values above 850 that were not isolated observations, and no data dictionary or other documentation was found to identify the specific scoring model used, these high credit scores were retained. Modifying these values without additional documentation could potentially lead to incorrect or invalid observations.

The summary statistics for the categorical variables were also generated to inspect the missing counts, unique categories, view the most frequent category, the frequency, relative frequency, and percent for each variable:

```
#Summary statistics for Categorical columns
cat_overview = []

for col in cat_cols:
    temp_col = df_keep[col].astype('object').fillna('Missing')
    value_counts = temp_col.value_counts()
    relative_freq = temp_col.value_counts(normalize = True)

    cat_overview.append([
        'Variable': col,
        'Non-Missing Count': df_keep[col].count(),
        'Missing Count': df_keep[col].isnull().sum(),
        'Missing Percent': (df_keep[col].isnull().sum() / len(df_keep)) * 100,
        'Unique Categories': df_keep[col].nunique(dropna=True),
        'Most Frequent Category': value_counts.index[0],
        'Frequency': value_counts.iloc[0],
        'Relative Frequency': relative_freq.iloc[0],
        'Percent': relative_freq.iloc[0] * 100})

cat_overview_df = pd.DataFrame(cat_overview)
cat_overview_df
```

	Variable	Non-Missing Count	Missing Count	Missing Percent	Unique Categories	Most Frequent Category	Frequency	Relative Frequency	Percent
0	credit_type	148670	0	0.000000	4	CIB	48152	0.323885	32.388511
1	loan_type	148670	0	0.000000	3	type1	113173	0.761236	76.123630
2	loan_purpose	148536	134	0.090133	4	p3	55934	0.376229	37.622923
3	loan_limit	145326	3344	2.249277	2	cf	135348	0.910392	91.039214
4	approv_in_adv	147762	908	0.610749	2	nopre	124621	0.838239	83.823905
5	Credit_Worthiness	148670	0	0.000000	2	I1	142344	0.957449	95.744938
6	business_or_commercial	148670	0	0.000000	2	nob/c	127908	0.860348	86.034842
7	Neg_ammortization	148549	121	0.081388	2	not_neg	133420	0.897424	89.742382
8	interest_only	148670	0	0.000000	2	not_int	141560	0.952176	95.217596
9	lump_sum_payment	148670	0	0.000000	2	not_lpsm	145286	0.977238	97.723818
10	age	148470	200	0.134526	7	45-54	34720	0.233537	23.353736
11	submission_of_application	148470	200	0.134526	2	to_inst	95814	0.644474	64.447434
12	occupancy_type	148670	0	0.000000	3	pr	138201	0.929582	92.958230
13	Region	148670	0	0.000000	4	North	74722	0.502603	50.260308

Once the data has been inspected, the next step is to perform basic cleaning on the dataset. A clean copy of the dataset was created, named `clean_df`:

```
#Creating cleaned copy of dataset
clean_df = df_keep.copy()
```

Any blank strings were replaced with missing values to ensure that all truly missing values were accounted for in the missing value counts:

```
# Replace blank strings with missing values
clean_df = clean_df.replace('', pd.NA)
```

The age variable, being treated as a string or object data type, must be converted to an ordered categorical variable, since the records indicate an incremental range of ages in years.

```
#Converting age to an ordered categorical variable
age_order = ['<25', '25-34', '35-44', '45-54', '55-64', '65-74', '>74']

clean_df['age'] = pd.Categorical(
    clean_df['age'],
    categories = age_order,
    ordered = True)

#Verifying age categories
clean_df['age'].value_counts(dropna = False).sort_index()

age
<25      1337
25-34    19142
35-44    32818
45-54    34720
55-64    32534
65-74    20744
>74       7175
NaN        200
Name: count, dtype: int64
```

To further preprocess the dataset, another copy was created for predictive modeling. The categorical variables were then standardized before splitting the data. The categorical predictors are converted before the preprocessing pipeline that imputes missing values and encodes the categorical variables. Furthermore, missing values were standardized to np.nan to ensure a consistent missing-value format across all categorical variables (GeeksforGeeks, 2025). Lastly, the target variable distribution is confirmed as a diagnostic check of the current class imbalance. This supports the need for stratified train-test splits during dataset splitting.

```
# Create a separate copy for predictive modeling
model_df = clean_df.copy()

#Convert categorical predictors to object type & represent missing values as np.nan
model_cat_columns = (model_df.drop(columns = 'Status').select_dtypes(include = ['object', 'string', 'category']).columns.tolist())

model_df[model_cat_columns] = (model_df[model_cat_columns].astype(object))

model_df[model_cat_columns] = (model_df[model_cat_columns].where(model_df[model_cat_columns].notna(), np.nan))

# Confirm target distribution
print(model_df['Status'].value_counts())

Status
0      112031
1       36639
Name: count, dtype: int64
```

The dataset is then split 70/15/15, with 70% for the training set, 15% for the validation set, and 15% for the test set. The dataset is split first, then missing values are imputed, and categorical variables are encoded to prevent data leakage. For predictive modeling, the data were split into subsets prior to fitting data-dependent preprocessing measures to prevent information from the evaluation subsets from influencing model training.

```
#Splitting the dataset 70/15/15
X = model_df.drop('Status', axis = 1)
y = model_df['Status'].astype(int)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size = 0.15, random_state = 42, stratify = y)

X_train, X_val, y_train, y_val = train_test_split(
    X_train, y_train, test_size = (15/85), random_state = 42, stratify = y_train)

#Quick check
print(X_train.shape, y_train.shape)
print(X_val.shape, y_val.shape)
print(X_test.shape, y_test.shape)

(104068, 20) (104068,)
(22301, 20) (22301,)
(22301, 20) (22301,)
```

The categorical and numeric variables were then identified and relabeled as `cat_features` and `num_features`, respectively, within the training data to prepare for the model processor.

```
#Identifying categorical and numeric predictors
cat_features = (
    X_train.select_dtypes(include = ['object', 'string', 'category'])
    .columns.tolist()

num_features = (
    X_train.select_dtypes(include = np.number).columns.tolist())

print('Categorical predictors:', cat_features)
print('Numeric predictors:', num_features)

Categorical predictors: ['credit_type', 'loan_type', 'loan_purpose', 'loan_limit', 'approv_in_adv', 'Credit_Worthiness', 'business_or_commercial', 'Neg_ammortization', 'interest_only', 'lump_sum_payment', 'age', 'submission_of_application', 'occupancy_type', 'Region']
Numeric predictors: ['loan_amount', 'term', 'income', 'Credit_Score', 'LTV', 'dtir1']
```

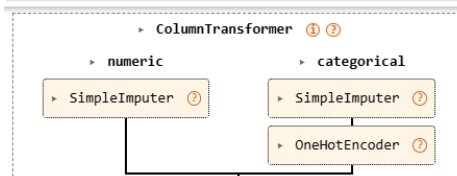
To handle missing values in the dataset and encode categorical variables, I implemented a pipeline that handles these preprocessing steps more smoothly. The `SimpleImputer` function was utilized, which supports median imputation for numeric variables and constant imputation with the label “Unknown” for categorical variables (`SimpleImputer`, n.d.). Furthermore, the `OneHotEncoder(handle_unknown = “ignore”)` helps minimize errors when test or future API data may contain categories that did not appear during training (Anandgupta, 2024).

```
#Numeric processing: replacing missing numeric values using training-set medians
num_pipeline = Pipeline(
    steps = [(
        'imputer', SimpleImputer(strategy = 'median'))])

#Categorical preprocessing: replacing missing categories and perform one-hot encoding
cat_pipeline = Pipeline(
    steps = [(('imputer', SimpleImputer(strategy = 'constant', fill_value = 'Unknown')),
        ('encoder', OneHotEncoder(handle_unknown = 'ignore')))])

#Applying transformations to numeric and categorical columns
preprocessor = ColumnTransformer(
    transformers = [(('numeric', num_pipeline, num_features),
        ('categorical', cat_pipeline, cat_features)), remainder = 'drop')

preprocessor
```



The preprocessor was then fitted using only the training data and subsequently applied to the testing data:

```
#Fitting preprocessed steps using training data
X_train_process = preprocessor.fit_transform(X_train)

#Applying training-fitted preprocessing rules to validation and test sets
X_val_process = preprocessor.transform(X_val)
X_test_process = preprocessor.transform(X_test)

print('Processed training shape:', X_train_process.shape)
print('Processed validation shape:', X_val_process.shape)
print('Processed testing shape:', X_test_process.shape)

Processed training shape: (104068, 53)
Processed validation shape: (22301, 53)
Processed testing shape: (22301, 53)
```

As a confirmation measure, a quality check was conducted to ensure that the missing values were indeed handled and that no missing values were present later.

```
#Confirming missing values are handled
from scipy import sparse

def count_miss_values(data):
    if sparse.issparse(data):
        return np.isnan(data.data).sum() #for sparse matrix
    return np.isnan(data).sum() #regular numpy array

print('Missing Values from Training set:', count_miss_values(X_train_process))
print('Missing Values from Validation set:', count_miss_values(X_val_process))
print('Missing Values from Testing set:', count_miss_values(X_test_process))

Missing Values from Training set: 0
Missing Values from Validation set: 0
Missing Values from Testing set: 0
```

Another check was conducted to confirm that the encoding process was successful by printing the feature names. Here, the total number of feature names should have increased to 53.

```
#Confirming feature names after encoding
process_feature_names = (preprocessor.get_feature_names_out())

print('Number of processed features:', len(process_feature_names))

print(process_feature_names)

Number of processed features: 53
['numeric_loan_amount' 'numeric_term' 'numeric_income'
 'numeric_Credit_Score' 'numeric_LTV' 'numeric_dti1'
 'categorical_credit_type_CIB' 'categorical_credit_type_CRIF'
 'categorical_credit_type_EQUI' 'categorical_credit_type_EXP'
 'categorical_loan_type_type1' 'categorical_loan_type_type2'
 'categorical_loan_type_type3' 'categorical_loan_purpose_Unknown'
 'categorical_loan_purpose_p1' 'categorical_loan_purpose_p2'
 'categorical_loan_purpose_p3' 'categorical_loan_purpose_p4'
 'categorical_loan_limit_Unknown' 'categorical_loan_limit_cf'
 'categorical_loan_limit_ncf' 'categorical_approv_in_adv_Unknown'
 'categorical_approv_in_adv_nopre' 'categorical_approv_in_adv_pre'
 'categorical_Credit_Worthiness_l1' 'categorical_Credit_Worthiness_l2'
 'categorical_business_or_commercial_b/c'
 'categorical_business_or_commercial_nob/c'
 'categorical_Neg_ammortization_Unknown'
 'categorical_Neg_ammortization_neg_amm'
 'categorical_Neg_ammortization_not_neg'
 'categorical_interest_only_int_only'
 'categorical_interest_only_not_int' 'categorical_lump_sum_payment_lpsm'
 'categorical_lump_sum_payment_not_lpsm' 'categorical_age_25-34'
 'categorical_age_35-44' 'categorical_age_45-54'
 'categorical_age_55-64' 'categorical_age_65-74' 'categorical_age_<25'
 'categorical_age_>74' 'categorical_age_Unknown'
 'categorical_submission_of_application_Unknown'
 'categorical_submission_of_application_not_inst'
 'categorical_submission_of_application_to_inst'
 'categorical_occupancy_type_ir' 'categorical_occupancy_type_pr'
 'categorical_occupancy_type_sr' 'categorical_Region_North'
 'categorical_Region_North-East' 'categorical_Region_central'
 'categorical_Region_south']
```

The final step of the data preparation phase was to save the cleaned dataset. X is transformed again to ensure that the numeric means and encoded categories still come only from the training set, and that the test observations do not influence preprocessing. The training-fitted transformer was applied to the complete predictor dataset to obtain a cleaned and encoded dataset ready for modeling. The final dataset, therefore, contains no missing values, and all categorical variables have been converted into numeric one-hot-encoded features required for a random forest classifier.

```
#Transforming complete predictor dataset
X_full_process = preprocessor.transform(X)

process_feature_names = preprocessor.get_feature_names_out()

#Converting transformed predictors into DataFrame
if sparse.issparse(X_full_process):
    clean_model_df = pd.DataFrame.sparse.from_spmatrix(
        X_full_process,
        index = X.index,
        columns = process_feature_names)
else:
    clean_model_df = pd.DataFrame(
        X_full_process,
        index = X.index,
        columns = process_feature_names)

#Adding back target (Status) variable
clean_model_df['Status'] = y.loc[
    clean_model_df.index].astype(int)

clean_model_df.to_csv('clean_rfmodel_loan_dataset.csv', index = False)
```

This transformed dataset was saved as a prepared dataset deliverable. The model was trained utilizing the training subset, and its performance was evaluated on the test set.

Tools and Techniques

Various tools and techniques have been utilized thus far in the analysis, including loading, inspecting, and cleaning the loan default data. The libraries and tools selected and utilized from Python are listed below, along with how each tool or technique supported the data extraction and data preparation process:

- **JupyterLab:** The data extraction and data preparation process took place using a JupyterLab environment. This allowed me to make sure my code was functioning properly and to provide a more user-friendly visual representation of the outputs. JupyterLab provides an interactive environment where Python code, markdown explanations, and visual outputs can all be in one notebook. An advantage of JupyterLab is that it supports rapid examination of outputs during the data extraction and preparation phase. I can view the dataset's structure, variables, records, and summary statistics directly in the code cells. However, a major disadvantage of this environment is that the

code cells can be executed out of order. If the code cells are executed out of order, the outputs may not accurately reflect the code's intended sequence. Therefore, it is always recommended to restart the kernel and rerun the code cells from the beginning to ensure that the outputs are accurate and free from errors.

- **Pandas (pandas):** The pandas library was the primary tool in the data extraction and preparation process. The pandas library helps me load the Excel file containing the data into a pandas DataFrame using the `pd.read_excel()` function. It also allows me to use functions to inspect the dataset's structure, data types, target distribution, missing values, duplicate rows, and other potential quality issues, such as `.shape()`, `.info()`, `isnull().sum()`, and `duplicated()`. Pandas also allows me to create working copies of the dataset using the `.copy()` function so that the original dataframe is not changed. Lastly, pandas was used to save the complete transformed dataset as a ready CSV deliverable. An advantage of using the pandas library is that it provides a lot of functions and tools for importing, inspecting, cleaning, summarizing, copying, and saving data. This was helpful since the loan default dataset contained various quality issues and variable types (numeric and categorical) that needed preparation. A disadvantage is that pandas often stores dataframes in memory, which can be resource-intensive, especially when working with very large datasets (Anand, 2023).
- **NumPy (numpy):** NumPy was utilized for numerical operations and promotes consistency in the representation of missing values. Categorical variables were converted to object type, and missing categorical observations were set to `np.nan` to standardize the input to scikit-learn preprocessing steps. Functions such as `np.isnan()` were also used to confirm that no missing values were present after the dataset was preprocessed. An advantage of NumPy is that it supports numerical operations and standardization of missing values. A disadvantage is that `np.nan` is mainly a floating-point value, and introducing it into some columns may affect their data types; it is also less descriptive than a specific label, such as "Unknown," when a missing value is in a categorical variable.
- **Scikit-learn:** The Scikit-learn library contained several tools, such as `train_test_split`, `ColumnTransformer`, `Pipeline`, `SimpleImputer`, and `OneHotEncoder`, to further prepare the dataset in different procedures and efficiently split the dataset prior to modeling. The `train_test_split()` function was utilized to split the data into training, validation, and test subsets before the preprocessing components were fitted. A stratified split approach was conducted since the target variable, Status, contains an unequal distribution between the default and non-default observations. `Random_state` allows for reproducible splitting. An advantage of using the `train_test_split()` function is that it keeps an independent test set for evaluating a trained model. A disadvantage is that a single train-test split yields only one estimate of performance, and the subsequent evaluation may be influenced by observations assigned to each subset.

- The **ColumnTransformer** function was used to combine the transformations of the numeric variables being processed to the numeric processing pipeline and the categorical variables being processed into the categorical processing pipeline into one feature matrix. This was done so that separate transformations are applied to different dataframe columns, and the results are then concatenated into one feature space (ColumnTransformer, n.d.). An advantage of this approach is that it enables the organized preparation of heterogeneous data into one object, ensuring that the same columns undergo the same transformations across the training, testing, and deployment phases. A disadvantage, however, is that columns that are not included in the transformer are dropped. Therefore, it is important to review the lists of numerical and categorical features to ensure that a predictor intended for the analysis is not accidentally omitted.
- The **SimpleImputer** function was used to handle missing values through the imputation approach. Missing values in the selected numeric columns were replaced with the calculated median, as several variables, such as income, loan amount, and debt-to-income ratio, contain extreme values. Such extreme values do not generally affect the median; therefore, the median was used during the imputation process. For the categorical variables, missing values were replaced with the category “Unknown” to retain observations in the dataset and distinguish unaccounted values from observed records. An advantage of this imputation method is that it allows variables with a certain portion of missing data to remain useful for the analysis. This is a safer approach than removing a large number of values, which could affect the model's analysis results. A disadvantage, however, is that imputation does not replace or recover the true missing values. Hence, the imputed values should be interpreted as functional replacements rather than verified information about the loan borrowers.
- **OneHotEncoder** was utilized during the data preparation process to convert the categorical variables into numeric binary features that will be provided to the Random Forest classifier. The encoder was set to `handle_unknown = 'ignore'` to allow future observations with a category not present during training to be transformed. An advantage of one-hot encoding is that it allows object or string categories to be converted into numeric predictors without assigning an artificial ranking to nominal variables. For instance, the various credit types were transformed into distinct indicators rather than assigned ordered numeric values. A disadvantage of one-hot encoding, however, is that it increases the number of features, creating a high-dimensional matrix that may increase computational requirements and resources and complicate the interpretation of feature associations.
- Scikit-learn's **Pipeline** function was utilized as a way to organize and streamline the imputing and encoding preprocessing steps. The numeric pipeline executed

the median imputation strategy on the selected numeric predictors, while the categorical pipeline handled missing values by replacing them with “Unknown” and then one-hot encoding all categorical variables. The pipeline chains estimators so that the output of one transformation becomes the input to the next step (Pipelines and composite estimators, n.d.). An advantage of using a pipeline method is reproducibility, in that the same preprocessing steps can be applied to the different subsets and new loan applications that come in after the model has been deployed. The preprocessor, along with the optimized model, can minimize the likelihood that deployment inputs will need to be prepared differently from the original training data. On the other hand, a disadvantage is that intermediate outputs are less noticeable within the pipelines. Debugging can require a more precise determination of where errors occurred and in which steps, inspecting transformed feature names, and taking more time to access individual steps.

- **SciPy Sparse-Matrix:** The sparse module within SciPy was utilized to see whether the transformed data was stored as a sparse matrix. Since one-hot encoding produced many binary values, the matrix contained many zeros. The sparse structures store mostly the values that are not zero and their locations. The `issparse()` function helps determine if an object is a sparse array or matrix (Issparse – SciPy v1.18.0 manual, n.d.). In this analysis, the function was used primarily for verification to ensure that missing values were correctly accounted for, thereby confirming the prepared output. An advantage of having a sparse representation is that it can minimize the memory that is needed to store matrices that have a large number of zeros that were created through one-hot encoding. However, a disadvantage is that sparse matrices may make it harder and slower to access individual points or elements (Ravihara, 2022).

Analysis

Exploratory Data Analysis

During the exploratory data analysis phase, visualizations were used to examine the data for trends and patterns. The visualizations were created and explored using the selected 21 variables from the dataset, including the Status target variable. Univariate, bivariate, and multivariate visualizations were created to analyze the distributions of the target variable, numeric variables, and categorical variables; examine potential patterns between each variable and the target variable; and identify possible trends or relationships between two or more variables and the target variable, respectively.

The first visualization below displays the distribution of the target variable of this analysis, Status. Based on the graph, substantially more borrowers have not defaulted on their loans than have. This demonstrates an unequal distribution of the target variable.

Distribution of Loan Default Status (target variable):



The next set of visualizations, displayed as histogram subplots, shows the distributions of the selected numeric variables (e.g., loan_amount, income, LTV, and dtir1). The histograms demonstrate that, for the most part, each numeric variable is skewed. For example, the loan amounts are right-skewed, with some extreme values. This can be a realistic situation, as some borrowers may take out large loan amounts. The borrowers' incomes are also highly right-skewed, indicating that some borrowers have very high incomes relative to the majority. The debt-to-income ratio is left-skewed, indicating some extremely low ratios. The loan-to-value ratio appears left-skewed in the histogram, but the visualization primarily shows a central distribution, with one extremely high value that ultimately makes the variable have an extreme right tail.

Numeric variable distributions:

```
#Univariate visualizations (histograms) of the numeric variable distributions
num_vars = {
    'loan_amount': 'Loan Amount',
    'income': 'Borrower Income',
    'LTV': 'Loan-to-Value Ratio',
    'dti': 'Debt-to-Income Ratio'
}

fig, axes = plt.subplots(2, 2, figsize = (15,8))

for ax, (column, label) in zip(axes.flatten(), num_vars.items()):
    plot_data = df_keep.loc[
        df_keep[column] <= df_keep[column].quantile(0.99), column]

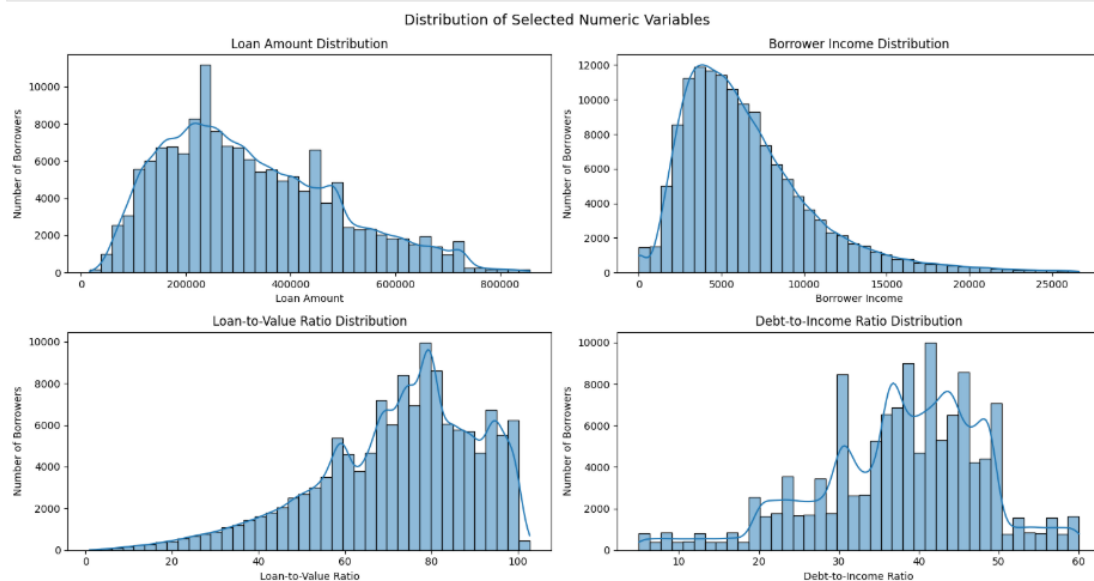
    sns.histplot(data = plot_data, bins = 40, kde = True, ax = ax)

    ax.set(
        title = f'{label} Distribution',
        xlabel = label,
        ylabel = 'Number of Borrowers')

plt.suptitle(
    'Distribution of Selected Numeric Variables',
    fontsize = 14)

plt.tight_layout()

plt.show()
```



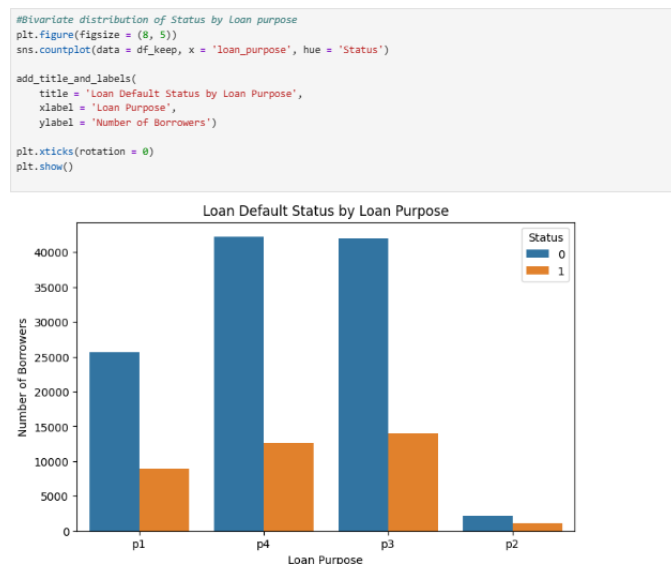
The distributions of a few selected categorical variables (e.g., credit_type, loan_type, age) were also displayed in subplots. For credit type, EQUI had the fewest borrowers, while CIB had the most. Most borrowers also had type 1 loans. The majority of observations are concentrated among borrowers aged 35 to 64.

Categorical variable distributions:



The first bivariate visualization demonstrates possible relationships between loan purpose and loan default status. The graph shows that the P2 category contained the largest number of default observations. Other loan purposes were somewhat similar to each other, with slight differences.

Loan Purpose vs. Loan Default Status:



Another bivariate visualization demonstrates a box plot displaying the credit score and loan default status variables. Based on the boxplots, the credit scores appear somewhat similar between those borrowers who have defaulted and those who have not. In other words, this demonstrates there is little visible difference between the groups.

Credit Score vs Loan Default Status:



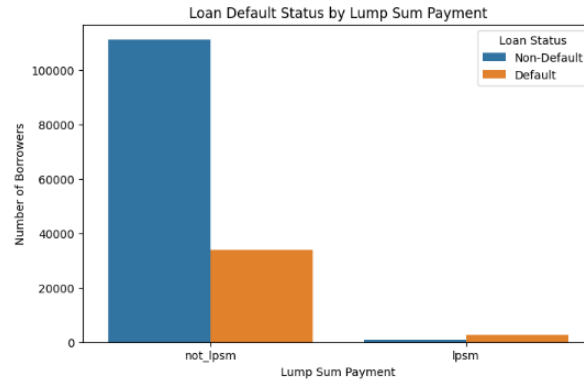
The next bivariate visualization, a double bar chart, displays the lump-sum payment variable and the loan default status variable. Among borrowers who have not made lump-sum payments, there were a higher number of non-default observations than the count of default observations. For borrowers who have made lump-sum payments, there is little difference in whether they defaulted, with fewer borrowers who made lump-sum payments having defaulted than those who did not.

Lump Sum Payment vs Loan Default Status:

```
#Bivariate visualization of status variable and Lump sum payment variable
plt.figure(figsize = (8, 5))
sns.countplot(data = df_keep, x = 'lump_sum_payment', hue = 'Status')

add_title_and_labels(
    title = 'Loan Default Status by Lump Sum Payment',
    xlabel = 'Lump Sum Payment',
    ylabel = 'Number of Borrowers')

plt.legend(title = 'Loan Status', labels = ['Non-Default', 'Default'])
plt.xticks(rotation = 0)
plt.show()
```



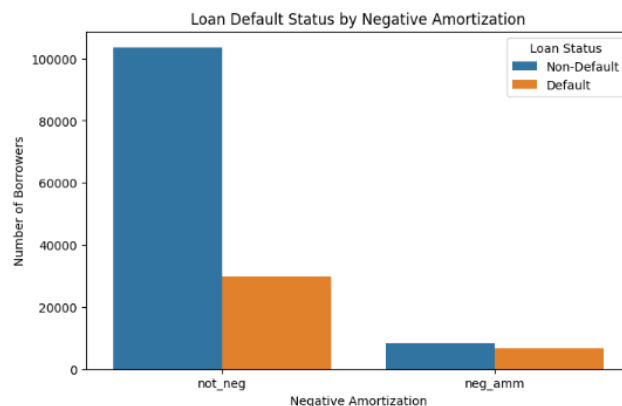
The next bivariate visualization is another double-bar chart showing the negative amortization variable and the loan default status variable. Among borrowers without negative amortization, there were fewer default observations than non-default observations. There was little difference, however, among borrowers with negative amortization, with slightly more borrowers who have not defaulted in this group.

Negative Amortization vs Loan Default Status:

```
#Bivariate visualization of status variable and negative ammortization variable
plt.figure(figsize = (8, 5))
sns.countplot(data = df_keep, x = 'Neg_ammortization', hue = 'Status')

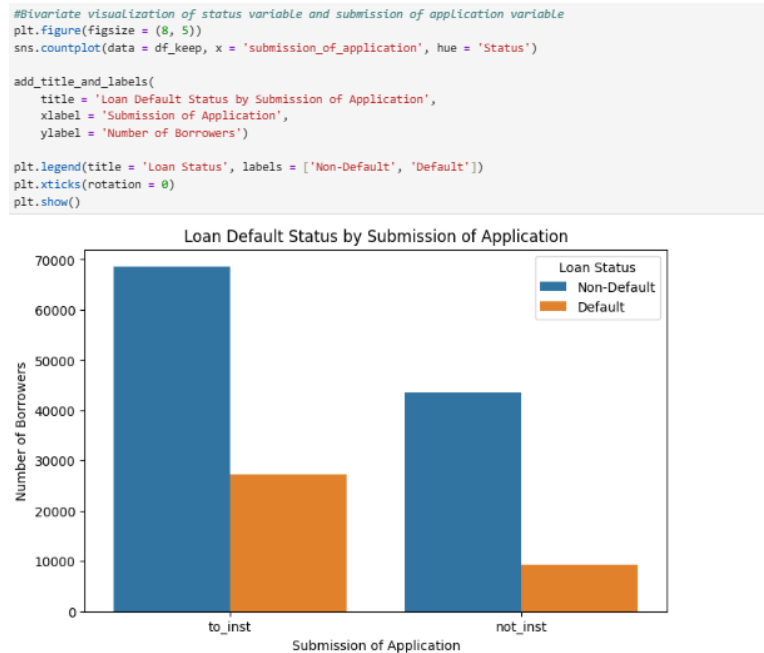
add_title_and_labels(
    title = 'Loan Default Status by Negative Amortization',
    xlabel = 'Negative Amortization',
    ylabel = 'Number of Borrowers')

plt.legend(title = 'Loan Status', labels = ['Non-Default', 'Default'])
plt.xticks(rotation = 0)
plt.show()
```



The double bar chart below shows the submissions for the application variable and the loan status variable. The raw counts indicate that there were more non-default observations in both application submission categories.

Submission of Application vs Loan Default Status:



The final bivariate visualization is a bar chart showing the age and loan default status variables. Visually, borrowers under 25 and over 74 have comparatively high rates of loan defaults.

Age vs Loan Default Status:

```

#Bivariate visualization of Status by age group
age_order = ['<25', '25-34', '35-44', '45-54', '55-64', '65-74', '>74']
age_default_rate = (df_keep.groupby('age')['Status'].mean().reindex(age_order).mul(100))

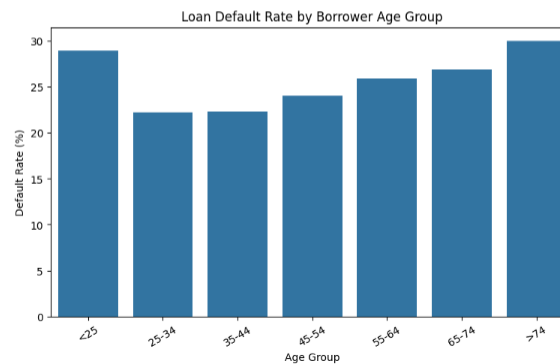
plt.figure(figsize=(9, 5))

sns.barplot(x = age_default_rate.index, y = age_default_rate.values)

add_title_and_labels(
    title = 'Loan Default Rate by Borrower Age Group',
    xlabel = 'Age Group',
    ylabel = 'Default Rate (%)')

plt.xticks(rotation=30)
plt.show()

```



A multivariate heatmap visualization displays the loan default rate by credit type and loan type. Borrowers with the EQUI credit type, regardless of loan type, exhibit higher default rates. Borrowers with other credit types, regardless of loan type, have lower rates of loan default. One thing to note is that visually, it appears that the EQUI credit type truly has no non-default cases.

Credit Type vs Loan Type vs Loan Default Status:

```

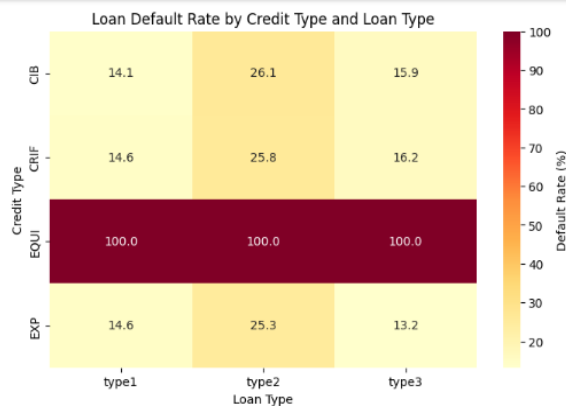
#Multivariate visualization (heatmap) of default rate by credit type and Loan type
loan_heatmap = (df_keep.pivot_table(index = 'credit_type', columns = 'loan_type', values = 'Status', aggfunc = 'mean').mul(100))

plt.figure(figsize = (8, 5))
sns.heatmap(loan_heatmap, annot = True, fmt = '.1f', cmap = 'YlOrRd', cbar_kws = {'label': 'Default Rate (%)'})

add_title_and_labels(
    title = 'Loan Default Rate by Credit Type and Loan Type',
    xlabel = 'Loan Type',
    ylabel = 'Credit Type')

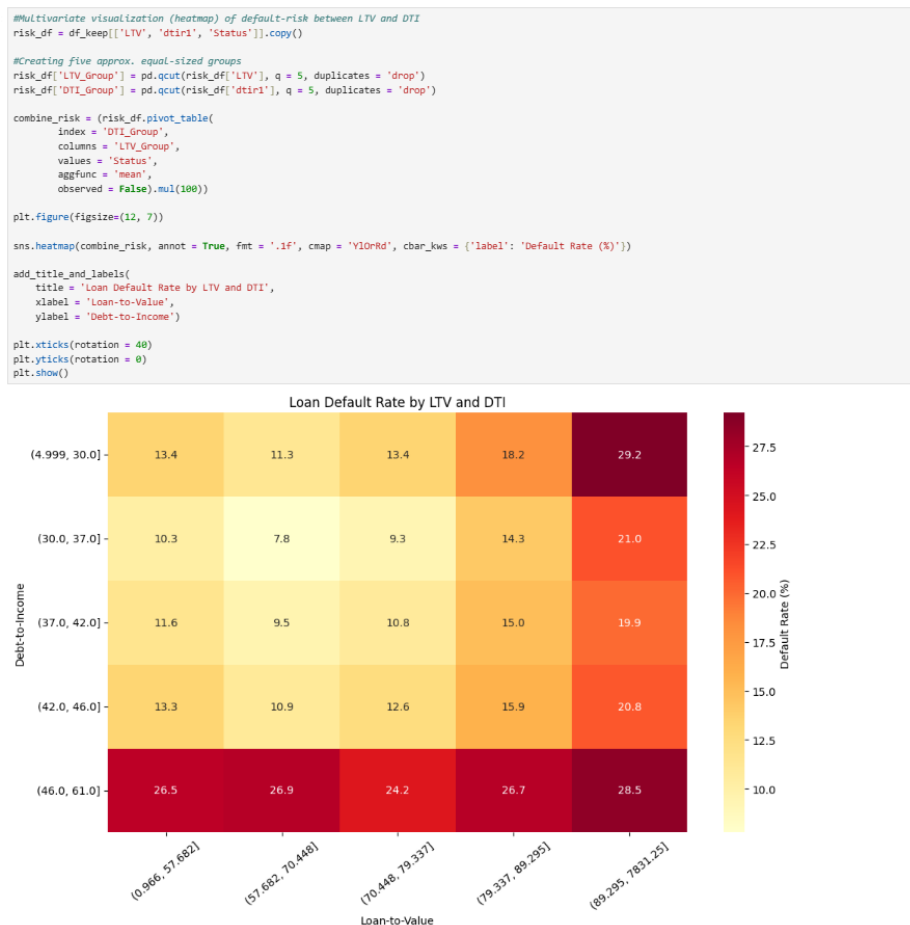
plt.show()

```



The second multivariate visualization, a heatmap, shows the loan default rate by LTV (loan-to-value ratio) and DTI (debt-to-income ratio). Borrowers who have higher DTI ratios, regardless of LTV ratios, show visibly higher default rates. Furthermore, borrowers with higher LTVs, regardless of DTI, also have higher default rates.

LTV vs DTI vs Loan Default Status:



An advantage of using exploratory visualizations during exploratory data analysis is that they provide a visual way to examine distributions, possible patterns, and extreme values before statistical testing and modeling. The visualizations in this analysis allowed me to identify skewed variables and examine default counts and rates across the categorical variables. A disadvantage here is that visualizations may be affected by axis limits, category-size imbalances, and binning choices.

Inferential Statistical Testing

Inferential statistics were conducted on the selected variables to assess the statistical significance of the Status variable. To prepare for this statistical analysis, the categorical variables were identified and evaluated using Chi-square tests of independence. Numeric variables were identified and evaluated using either Welch's independent-samples t-test or the Mann-Whitney U test based on the distributions of the observations between the default and non-default groups. The alpha level was also set to 0.05.

```
#Categorical variables for Chi-Square independence testing
categor_cols = [
    'credit_type',
    'loan_type',
    'loan_purpose',
    'loan_limit',
    'approv_in_adv',
    'credit_worthiness',
    'business_or_commercial',
    'Neg_amortization',
    'interest_only',
    'lump_sum_payment',
    'age',
    'submission_of_application',
    'occupancy_type',
    'Region']

#Numeric variables for Mann-Whitney U tests
numeric_cols = [
    'loan_amount',
    'term',
    'income',
    'Credit_Score',
    'LTV',
    'dtirs']

#Significance Level
alpha = 0.05
```

The Chi-square test was performed on the selected categorical variables. A dataframe was created to display the results of the test for each variable, including the Chi-Square statistic, degrees of freedom, p-values, Cramer's v, and a Boolean result of whether the variable is significant at 0.05. The Chi-square results indicate that all selected categorical variables had very small p-values, well below the significance level. Therefore, all categorical variables are significant at the 0.05 level; hence, the null hypothesis for each variable and the loan default status was rejected. This results in evidence of an association, but not of causation or of the direction of the association.

Chi-Square Independence Test:

```
#Chi-square test
chi_square_results = []

for col in category_cols:
    table = pd.crosstab(clean_df[col], clean_df['Status'])

    chi2, p_value, dof, _ = chi2_contingency(table)

    n = table.to_numpy().sum()
    min_dim = min(table.shape) - 1

    crammers_v = (
        np.sqrt(chi2 / (n * min_dim))
        if min_dim > 0
        else np.nan
    )

    chi_square_results.append([
        'Variable': col,
        'Chi-square Statistic': chi2,
        'Degrees of Freedom': dof,
        'P-value': p_value,
        'Cramer's V': crammers_v,
        'Significant at 0.05': p_value < alpha])

chi_square_df = (pd.DataFrame(chi_square_results).sort_values('P-value').reset_index(drop = True))

chi_square_df = chi_square_df.round({'Chi-square Statistic': 3, 'Cramer's V': 3})

chi_square_df
```

	Variable	Chi-square Statistic	Degrees of Freedom	P-value	Cramer's V	Significant at 0.05
0	credit_type	52135.281	3	0.000000e+00	0.592	True
1	Neg_ammortization	3610.208	1	0.000000e+00	0.156	True
2	submission_of_application	2171.710	1	0.000000e+00	0.121	True
3	lump_sum_payment	5237.827	1	0.000000e+00	0.188	True
4	loan_type	1309.958	2	3.517253e-285	0.094	True
5	business_or_commercial	1272.808	1	9.172191e-279	0.093	True
6	loan_limit	427.398	1	5.985647e-95	0.054	True
7	Region	380.456	3	3.786057e-82	0.051	True
8	age	374.502	6	8.442234e-78	0.050	True
9	loan_purpose	240.214	3	8.559940e-52	0.040	True
10	approv_in_adv	206.402	1	8.374763e-47	0.037	True
11	Credit_Worthiness	180.421	1	3.922570e-41	0.035	True
12	occupancy_type	131.825	2	2.368574e-29	0.030	True
13	interest_only	28.495	1	9.395377e-08	0.014	True

One advantage of utilizing the Chi-square test of independence is that it is an appropriate method for assessing whether categorical variables, in this case, for example, a categorical variable on the loan default status variable, are statistically related without needing a normal distribution (McHugh, 2013). A disadvantage, however, is that associations do not equal causation. Furthermore, smaller associations can yield very low p-values in a larger dataset; hence, an additional measure, such as Cramer's V, is needed for further evaluation of practical importance (McHugh, 2013).

For the selected numeric variables, I first inspected each variable's skewness. This was another way to assess skewness after visually examining the histograms during the exploratory data analysis. In the results displayed below, all but two variables, credit score and debt-to-income ratio, have some degree of skewness and are therefore evaluated using the Mann-Whitney U test. The Credit_Score and dtir1 variables were approximately symmetric based on the skewness criterion and, therefore, will be evaluated using the Welch's t-test to compare the group means. This will be important to consider as different statistical tests will be performed based on the symmetry levels of the numeric variables.

```
#Checking skewness of numeric variables by Status group
skewness_res = []

for col in numeric_cols:
    group_0 = clean_df[clean_df['Status'] == 0][col].dropna()
    group_1 = clean_df[clean_df['Status'] == 1][col].dropna()

    skewness_res.append({
        'Variable': col,
        'Status 0 Skewness': group_0.skew(),
        'Status 1 Skewness': group_1.skew(),
        'Approximately Normal': abs(group_0.skew()) <= 1 and abs(group_1.skew()) <= 1 })

skewness_df = pd.DataFrame(skewness_res)

skewness_df
```

	Variable	Status 0 Skewness	Status 1 Skewness	Approximately Normal
0	loan_amount	1.216433	2.494070	False
1	term	-2.171591	-2.183568	False
2	income	14.911745	21.737594	False
3	Credit_Score	0.006499	-0.000710	True
4	LTV	123.105429	47.337365	False
5	dtir1	-0.576422	-0.647411	True

Based on whether a variable is approximately symmetric or not, either a Welch's t-test or a Mann-Whitney U test was performed. The results of both tests are displayed in a dataframe, including the test used, the means and medians for both groups, skewness values, the test statistic, p-values, and a Boolean indicating whether each variable is statistically significant. The credit score and debt-to-income ratio variables were approximately symmetric after checking for skewness, and therefore, a Welch's t-test was performed on them. Their means were similar between groups 0 and 1, as were their median values. The debt-to-income ratio had a low p-value below the threshold, indicating that the null hypothesis can be rejected and we can conclude that it is statistically significant. However, the credit score value had a p-value higher than the alpha threshold. Therefore, we cannot reject the null hypothesis for this variable and say that it is statistically significant.

The remaining selected numeric variables (e.g., income, LTV, loan amount, term) were skewed; therefore, the Mann-Whitney U test was conducted to evaluate differences in rank distributions. The p-values for all of these variables were very small, well below the alpha threshold. For these variables, the null hypothesis is rejected, and we conclude that they are statistically significant.

Welch's T-Test and Mann-Whitney U Test:

```

#Significance testing for numeric variables
num_test_results = []

for col in numeric_cols:
    group_0 = clean_df.loc[clean_df['Status'] == 0, col].dropna()

    group_1 = clean_df.loc[clean_df['Status'] == 1, col].dropna()

    skew_0 = group_0.skew()
    skew_1 = group_1.skew()

    #Using Welch's t-test when both groups are symmetric
    if max(abs(skew_0), abs(skew_1)) <= 1:
        test_name = "Welch's t-test"

        test_stat, p_value = ttest_ind(group_0, group_1, equal_var = False)

    #Using Mann-Whitney U for strongly skewed distributions
    else:
        test_name = 'Mann-Whitney U test'

        test_stat, p_value = mannwhitneyu(group_0, group_1, alternative = 'two-sided')

    num_test_results.append({
        'Variable': col,
        'Test Used': test_name,
        'Group 0 Mean': group_0.mean(),
        'Group 1 Mean': group_1.mean(),
        'Group 0 Median': group_0.median(),
        'Group 1 Median': group_1.median(),
        'Group 0 Skewness': skew_0,
        'Group 1 Skewness': skew_1,
        'Test Statistic': test_stat,
        'P-value': p_value,
        f'Significant at (alpha)': p_value < alpha})

sig_test_df = (pd.DataFrame(num_test_results).sort_values('P-value').reset_index(drop=True))

# Round descriptive values but preserve exact p-values
round_columns = [
    'Group 0 Mean',
    'Group 1 Mean',
    'Group 0 Median',
    'Group 1 Median',
    'Group 0 Skewness',
    'Group 1 Skewness',
    'Test Statistic']

sig_test_df[round_columns] = (
    sig_test_df[round_columns].round(3))

sig_test_df

```

	Variable	Test Used	Group 0 Mean	Group 1 Mean	Group 0 Median	Group 1 Median	Group 0 Skewness	Group 1 Skewness	Test Statistic	P-value	Significant at 0.05
0	income	Mann-Whitney U test	7204.014	6231.807	6000.000	4860.000	14.912	21.738	2.183041e+09	0.000000e+00	True
1	LTV	Mann-Whitney U test	72.065	76.291	74.504	79.362	123.105	47.337	1.035994e+09	6.021924e-238	True
2	loan_amount	Mann-Whitney U test	334990.775	319275.185	306500.000	276500.000	1.216	2.494	2.241604e+09	3.037459e-155	True
3	dtirl	Welch's t-test	37.369	39.597	38.000	42.000	-0.576	-0.647	-2.359100e+01	9.590572e-122	True
4	term	Mann-Whitney U test	335.145	335.112	360.000	360.000	-2.172	-2.184	2.082543e+09	4.521676e-11	True
5	Credit_Score	Welch's t-test	699.524	700.600	699.000	700.000	0.006	-0.001	-1.538000e+00	1.239889e-01	False

Inferential statistical testing was performed using the available non-null values for each variable. The observations missing for the variables tested were excluded from that specific test. Therefore, the sample sizes may vary across the tests. The imputation methods used to handle missing values were applied prior to the predictive modeling process, so that missing observations could be included in the modeling and evaluation process.

An advantage of using such inferential statistical tests is that they often help in determining whether the variables and the target variable are statistically associated. A major disadvantage, however, is that statistical significance does not imply direct causation. One advantage of utilizing Welch's independent-samples t-test during inferential statistical testing for numeric variables is that it does not require groups to have equal variances (Kim, 2019). This can be helpful since both groups, default and non-default, do not have equal amounts of observations and therefore can have different variances. A disadvantage is that the test focuses primarily on mean differences, which can be influenced by outliers. Regarding the Mann-Whitney U test, an advantage is that it is particularly useful when groups are skewed and therefore does not require

the normality assumption (McClenaghan, 2024). A disadvantage is that it tests whether rank distributions differ and should not be quickly translated as testing for median differences unless the groups' distributions are similar in shape (Frost, 2023).

Random Forest Model

The initial model was created using the processed trained dataset. A Random Forest Model was chosen as the primary modeling method for this analysis because it can leverage various dataset factors, such as borrower and credit characteristics, to predict whether a borrower is likely to default on a loan. A Random Forest classification method also provides feature importance values, which can help stakeholders and banks understand which factors are most strongly associated with loan default prediction. The screenshot below shows how the initial model was created, with the random state set to 42, class weights set to balanced, and n_jobs set to -1. The model was then evaluated on the test data, and the following metrics were reported: accuracy, ROC-AUC, precision, recall, F1 score, and confusion matrix.

Initial Random Forest Model:

```
#Instantiating and fitting initial model
rf = RandomForestClassifier(random_state = 42,
                           class_weight = 'balanced',
                           n_jobs = -1)

rf.fit(X_train_process, y_train)
```

RandomForestClassifier	
Parameters	
n_estimators	100
criterion	'gini'
max_depth	None
min_samples_split	2
min_samples_leaf	1
min_weight_fraction_leaf	0.0
max_features	'sqrt'
max_leaf_nodes	None
min_impurity_decrease	0.0
bootstrap	True
oob_score	False
n_jobs	-1
random_state	42
verbose	0
warm_start	False
class_weight	'balanced'
ccp_alpha	0.0
max_samples	None
monotonic_cst	None

Evaluating on Test Data (along with Accuracy and AUC Metrics):

```
#Evaluating on test data
y_test_pred_init = rf.predict(X_test_process)
y_test_proba_init = rf.predict_proba(X_test_process)[:, 1]

#Accuracy and AUC-ROC
print('Accuracy:', accuracy_score(y_test, y_test_pred_init))
print('AUC-ROC:', roc_auc_score(y_test, y_test_proba_init))

Accuracy: 0.8931886462490471
AUC-ROC: 0.8864676406351301
```

Precision, Recall, F1 Score Through Classification Report, and Confusion Matrix:

```
#Precision, recall, and F1 score using classification report
print('Classification Report:')
print(classification_report(y_test, y_test_pred_init))

Classification Report:
              precision    recall  f1-score   support

     0       0.88        0.99        0.93       16805
     1       0.94        0.61        0.74        5496

 accuracy          0.91        0.80        0.89       22301
 macro avg          0.91        0.80        0.84       22301
 weighted avg          0.90        0.89        0.88       22301

#Confusion matrix
print('Confusion Matrix:')
print(confusion_matrix(y_test, y_test_pred_init))

Confusion Matrix:
[[16581  224]
 [ 2158 3338]]
```

An advantage of using a Random Forest classifier for this analysis is that it can capture nonlinear relationships and interactions between variables. It is especially helpful for datasets with a mix of variable types, such as the current data for this analysis. A disadvantage, on the other hand, is that a Random Forest classifier can be less directly interpretable than simpler models (GeeksforGeeks, 2025).

Hyperparameter tuning was then performed to optimize the initial Random Forest model. The selected hyperparameters were the following: `n_estimators`, `max_depth`, `min_samples_split`, `min_samples_leaf`, `max_features`, and `class_weight`. The `RandomizedSearchCV` method was utilized with 3-fold cross-validation and fitted using the training dataset. Despite the split data containing a validation subset, it was not used in the final model comparison as `RandomizedSearchCV` performed internal cross-validation on the training dataset. The F1-score was the metric used for tuning due to the loan status variable's imbalanced class distribution. Moreover, it balances precision and recall for the default class.

The `n_estimators` hyperparameter was chosen because increasing the number of trees helps minimize variance and stabilize the ensemble (Part 6: random forest hyperparameters & tuning strategies, 2025). The `max_depth` hyperparameter was selected to control the levels of each decision tree. The `min_samples_split` hyperparameter was also selected to prevent the model from creating overly specific rules based on small groups of borrowers. The `min_samples_leaf` hyperparameter was also selected to manage how many samples a leaf node should have, thereby helping prevent overfitting. The `max_features` hyperparameter was selected

to broaden the diversity of the trees and thereby improve generalization by managing the number of predictor variables considered for each tree. Finally, the `class_weight` hyperparameter was chosen because the Status target variable is currently imbalanced; therefore, it can help the model assign more weight to the minority class or the default class to improve performance. The image below demonstrates the best hyperparameters selected.

Hyperparameter Tuning (Optimization):

```
#Hyperparameter options
from sklearn.model_selection import StratifiedKFold

#One processor per forest so RSCV can parallelize individual model fits
tune_rf = RandomForestClassifier(random_state = 42, n_jobs = 1)

param_opt = {
    'n_estimators': [150, 250, 350],
    'max_depth': [None, 15, 25, 35],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4],
    'max_features': ['sqrt', 'log2'],
    'class_weight': ['balanced', None]}

cv_strat = StratifiedKFold(n_splits = 3, shuffle = True, random_state = 42)

#Randomized search using 3-fold cross-validation
ran_search = RandomizedSearchCV(
    estimator = tune_rf,
    param_distributions = param_opt,
    n_iter = 15,
    cv = cv_strat,
    scoring = 'f1',
    random_state = 42,
    n_jobs = -1,
    verbose = 2,
    refit = True,
    return_train_score = False)

#Fitting random search on training data
ran_search.fit(X_train_process, y_train)

#Displaying best hyperparameters and cross-validation f1-score
print(ran_search.best_params_)
print(ran_search.best_score_)

Fitting 3 folds for each of 15 candidates, totalling 45 fits
{'n_estimators': 150, 'min_samples_split': 2, 'min_samples_leaf': 4, 'max_features': 'sqrt', 'max_depth': 35, 'class_weight': 'balanced'}
0.7378937526643399
```

An advantage of using the `RandomizedSearchCV` method for hyperparameter tuning is that it searches for hyperparameter combinations via cross-validation more efficiently than an exhaustive grid search. A disadvantage of this technique is that it does not test every possible combination; therefore, the best possible hyperparameters could be missed.

The optimized model was fitted, and predictions were generated on the test dataset. The images below show the optimized model fitted, along with the following metrics: accuracy, precision, F1 score, AUC-ROC, and a confusion matrix.

Optimized Model:

```
#Optimized model
optim_rf = ran_search.best_estimator_
optim_rf.fit(X_train_process, y_train)
```

RandomForestClassifier	
Parameters	
n_estimators	150
criterion	'gini'
max_depth	35
min_samples_split	2
min_samples_leaf	4
min_weight_fraction_leaf	0.0
max_features	'sqrt'
max_leaf_nodes	None
min_impurity_decrease	0.0
bootstrap	True
oob_score	False
n_jobs	1
random_state	42
verbose	0
warm_start	False
class_weight	'balanced'
ccp_alpha	0.0
max_samples	None
monotonic_cst	None

Predictions Made on Test Dataset and Evaluation Metrics Displayed:

```
#Predictions on test dataset
y_test_pred_opt = optim_rf.predict(X_test_process)
y_test_proba_opt = optim_rf.predict_proba(X_test_process)[:, 1]
```

```
#Displaying performance metrics of optimized model
print('Accuracy:', accuracy_score(y_test, y_test_pred_opt))
print('AUC_ROC:', roc_auc_score(y_test, y_test_proba_opt))

print('Classification Report:')
print(classification_report(y_test, y_test_pred_opt))

print('Confusion Matrix:')
print(confusion_matrix(y_test, y_test_pred_opt))
```

```
Accuracy: 0.8894219998134972
AUC_ROC: 0.8915484665852986
Classification Report:
              precision    recall  f1-score   support

      0       0.90      0.96      0.93      16885
      1       0.84      0.68      0.75       5496

   accuracy      0.89      0.89      0.89      22381
  macro avg      0.87      0.82      0.84      22381
 weighted avg      0.89      0.89      0.89      22381

Confusion Matrix:
[[16891   714]
 [ 1752  3744]]
```

Feature importance is another important aspect of the Random Forest classifier, as it allows us to examine variables that have higher importance to indicate predictive contribution. Using the `feature_importances_` function, the variables were sorted in descending order of importance

(Feature importance with a forest of trees, n.d.). A dataframe has been created to organize the results in a more user-friendly manner. Here, the credit type EQUI is an encoded category-level feature that appears to have higher importance for loan default predictions, followed by LTV, dtir1, income, and so on.

Feature Importance:

```
feature_import = pd.DataFrame({
    'Feature': process_feature_names,
    'Importance': optim_rf.feature_importances_
}).sort_values(by = 'Importance', ascending = False)
feature_import.head(10)
```

	Feature	Importance
8	categorical__credit_type_EQUI	0.200077
4	numeric__LTV	0.158179
5	numeric__dtir1	0.133111
2	numeric__income	0.081628
0	numeric__loan_amount	0.053515
3	numeric__Credit_Score	0.049013
6	categorical__credit_type_CIB	0.027282
7	categorical__credit_type_CRIF	0.025861
9	categorical__credit_type_EXP	0.023503
33	categorical__lump_sum_payment_lpsm	0.018928

Once the optimized model has been evaluated, it is now ready for a local prototype deployment demonstration. The images below show the preprocessor used to preprocess the data, and the final model is saved as a .joblib file for deployment. Verification steps have also been conducted to ensure that the deployment bundle can successfully load and be tested by generating a sample prediction.

Saving Preprocessor and Final Model for Deployment:

```
deployment_bund = {
    'preprocessor': preprocessor,
    'model': optim_rf,
    'raw_feature_names': X_train.columns.tolist(),
    'numeric_features': num_features,
    'categorical_features': cat_features
}

joblib.dump(
    deployment_bund, 'loan_model_bundle.joblib', compress = 3)

print('Deployment bundle successfully saved.')
Deployment bundle successfully saved.
```

Verifying the Deployment Bundle Can be Loaded and Testing Sample Prediction:

```

#Reloading and testing saved bundle
load_bundle = joblib.load('loan_model_bundle.joblib')

required_items = {
    'preprocessor',
    'model',
    'raw_feature_names'}

assert required_items.issubset(load_bundle.keys())

sample_index = 1
sample_in = (X_test.iloc[[sample_index]].copy())

#Verifying columns are in expected order
sample_in = sample_in[load_bundle['raw_feature_names']]

#Applying saved preprocessing rules
sample_process = (load_bundle['preprocessor']).transform(sample_in)

#Predicted class
sample_pred = int(load_bundle['model'].predict(sample_process)[0])

#Generating probability of Status = 1
sample_prob = float(load_bundle['model'].predict_proba(sample_process)[0, 1])

#Risk category
risk_category = (
    'Higher default risk'
    if sample_pred == 1
    else 'Lower default risk')

print('Default probability:', sample_prob)
print('Predicted Status:', sample_pred)
print('Risk Category:', risk_category)
print('Actual_Status:', int(y_test.iloc[sample_index]))

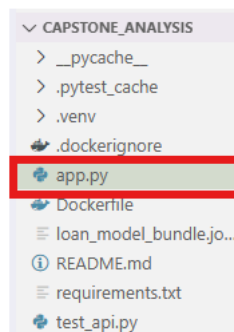
Default probability: 0.29401118343635835
Predicted Status: 0
Risk Category: Lower default risk
Actual_Status: 0

```

Deployment

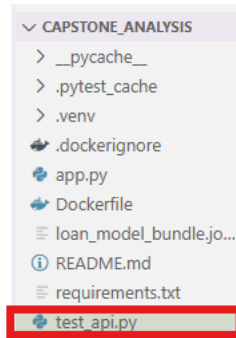
FastAPI Application:

Once the model had been saved as a .joblib file, a new folder was created in my interactive VS Code environment to locally deploy the model and make real-time predictions about loan defaults. A virtual environment was created. Then, a file named app.py was created containing the FastAPI application. Included are the request schema, the model loading process, the model inputs, and the prediction endpoints.



API Unit Tests:

To ensure that the API endpoints' features are tested, four unit tests were implemented to handle both correctly and incorrectly formatted requests, and they are located in the test_api.py file.



The first unit test simply tests that the API status endpoint returns a successful response. Here, we expect a status code of 200.

```
#Testing the API status endpoint returns a successful response
def test_api_status_endpoint():
    response = client.get("/")

    assert response.status_code == 200
    assert "message" in response.json()
```

The next unit test verifies that a correctly formatted request is accepted by a prediction endpoint.

```
#Testing that the prediction endpoint accepts a correctly formatted request
def test_predict_default_valid_request():
    response = client.post(
        "/predict/default-risk",
        json = {
            'loan_amount': 116500,
            'term': 360,
            'income': 1740,
            'Credit_Score': 758,
            'LTV': 98.72881356,
            'dtir1': 45,
            'credit_type': 'EXP',
            'loan_type': 'type1',
            'loan_purpose': 'p1',
            'loan_limit': 'cf',
            'approv_in_adv': 'nopre',
            'Credit_Worthiness': 'l1',
            'business_or_commercial': 'nob/c',
            'Neg_ammortization': 'not_neg',
            'interest_only': 'not_int',
            'lump_sum_payment': 'not_lpsm',
            'age': '25-34',
            'submission_of_application': 'to_inst',
            'occupancy_type': 'pr',
            'Region': 'south'})

    assert response.status_code == 200

    data = response.json()

    assert 'default_probability' in data
    assert 'predict_status' in data
    assert 'risk_category' in data
```

The third unit test is for an incorrectly formatted request in which the loan amount variable is misspelled. Hence, a 422 error is expected to be returned here.

```
#Testing an incorrectly formatted request returns a 422 error (Loan amount misspelled)
def test_predict_invalid_request():
    response = client.post(
        "/predict/default-risk",
        json = {
            'loan_amnt': 116500,
            'term': 360,
            "income": 1740,
            "Credit_Score": 758,
            "LTV": 98.72881356,
            "dtir1": 45,
            "credit_type": "EXP",
            "loan_type": "type1",
            "loan_purpose": "p1",
            "loan_limit": "cf",
            "approv_in_adv": "nopre",
            "Credit_Worthiness": "l1",
            "business_or_commercial": "nob/c",
            "Neg_ammortization": "not_neg",
            "interest_only": "not_int",
            "lump_sum_payment": "not_lpsm",
            "age": "25-34",
            "submission_of_application": "to_inst",
            "occupancy_type": "pn",
            "Region": "south"})

    assert response.status_code == 422
```

The last unit test is for another incorrectly formatted request that is missing the variable occupancy_type. A 422 error is also expected to be returned here.

```
#Testing a missing entry in request, returnning a 422 error (missing occupancy_type)
def test_predict_missing_request():
    response = client.post(
        "/predict/default-risk",
        json = {
            'loan_amount': 116500,
            'term': 360,
            "income": 1740,
            "Credit_Score": 758,
            "LTV": 98.72881356,
            "dtir1": 45,
            "credit_type": "EXP",
            "loan_type": "type1",
            "loan_purpose": "p1",
            "loan_limit": "cf",
            "approv_in_adv": "nopre",
            "Credit_Worthiness": "l1",
            "business_or_commercial": "nob/c",
            "Neg_ammortization": "not_neg",
            "interest_only": "not_int",
            "lump_sum_payment": "not_lpsm",
            "age": "25-34",
            "submission_of_application": "to_inst",
            "Region": "south"})

    assert response.status_code == 422
```

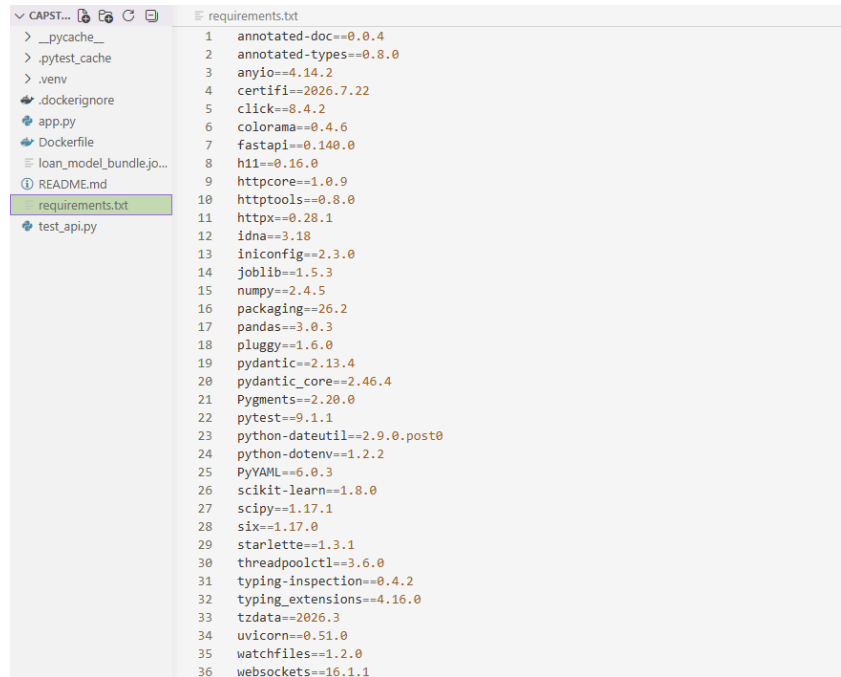
The image below demonstrates that all four unit tests passed successfully after running `pytest -v` in the terminal.

```
test_api.py::test_api_status_endpoint PASSED
test_api.py::test_predict_default_valid_request PASSED
test_api.py::test_predict_invalid_request PASSED
test_api.py::test_predict_missing_request PASSED
```

```
===== 4 passed in 3.98s =====
(.venv) PS C:\Users\katie\Capstone_analysis>
```

Requirements:

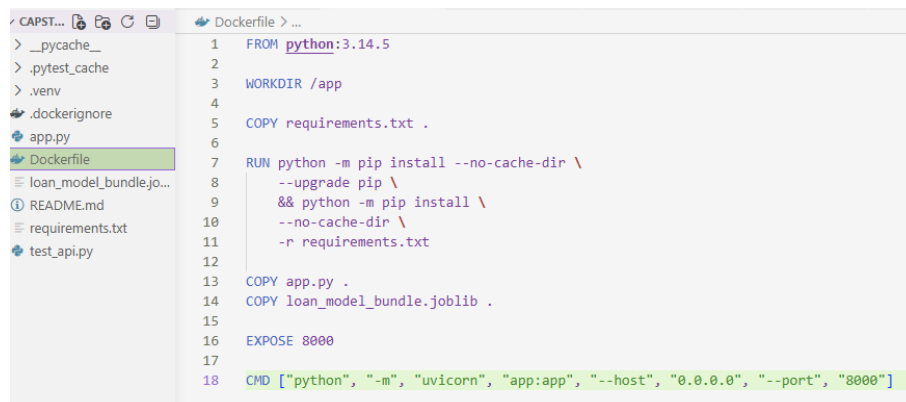
The requirements.txt file was created, which contains the list of Python packages required to run the application.



```
1 annotated-doc==0.0.4
2 annotated-types==0.8.0
3 anyio==4.14.2
4 certifi==2026.7.22
5 click==8.4.2
6 colorama==0.4.6
7 fastapi==0.140.0
8 h11==0.16.0
9 httpcore==1.0.9
10 httptools==0.8.0
11 httpx==0.28.1
12 idna==3.18
13 iniconfig==2.3.0
14 joblib==1.5.3
15 numpy==2.4.5
16 packaging==26.2
17 pandas==3.0.3
18 pluggy==1.6.0
19 pydantic==2.13.4
20 pydantic_core==2.46.4
21 Pygments==2.20.0
22 pytest==9.1.1
23 python-dateutil==2.9.0.post0
24 python-dotenv==1.2.2
25 PyYAML==6.0.3
26 scikit-learn==1.8.0
27 scipy==1.17.1
28 six==1.17.0
29 starlette==1.3.1
30 threadpoolctl==3.6.0
31 typing-inspection==0.4.2
32 typing_extensions==4.16.0
33 tzdata==2026.3
34 uvicorn==0.51.0
35 watchfiles==1.2.0
36 websockets==16.1.1
```

Dockerfile:

The Dockerfile was created to package the API code and run the web server to allow HTTP requests to the API.



```
1 FROM python:3.14.5
2
3 WORKDIR /app
4
5 COPY requirements.txt .
6
7 RUN python -m pip install --no-cache-dir \
8     --upgrade pip \
9     && python -m pip install \
10     --no-cache-dir \
11     -r requirements.txt
12
13 COPY app.py .
14 COPY loan_model_bundle.joblib .
15
16 EXPOSE 8000
17
18 CMD ["python", "-m", "uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000"]
```

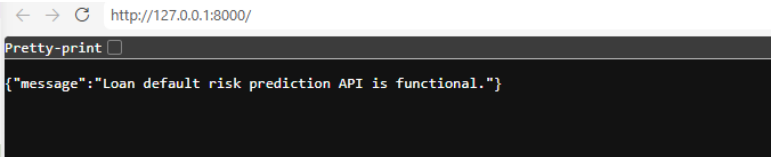
Prediction Endpoints:

To test my endpoints, a Docker image was built. Then, I ran the Docker container by running the following on the terminal: `docker run --rm --name loan-default-api-container -p 8000:8000 loan-default-api`. The Docker container exposes port 8000, mapping it between the host computer and the container. The image below demonstrates the status endpoint being tested, in which it successfully states that the loan default prediction API is functional:

```
(.venv) PS C:\Users\katie\Capstone_analysis> Invoke-RestMethod `
>> -Method Get `
>> -Uri "http://127.0.0.1:8000/"

message
-----
Loan default risk prediction API is functional.

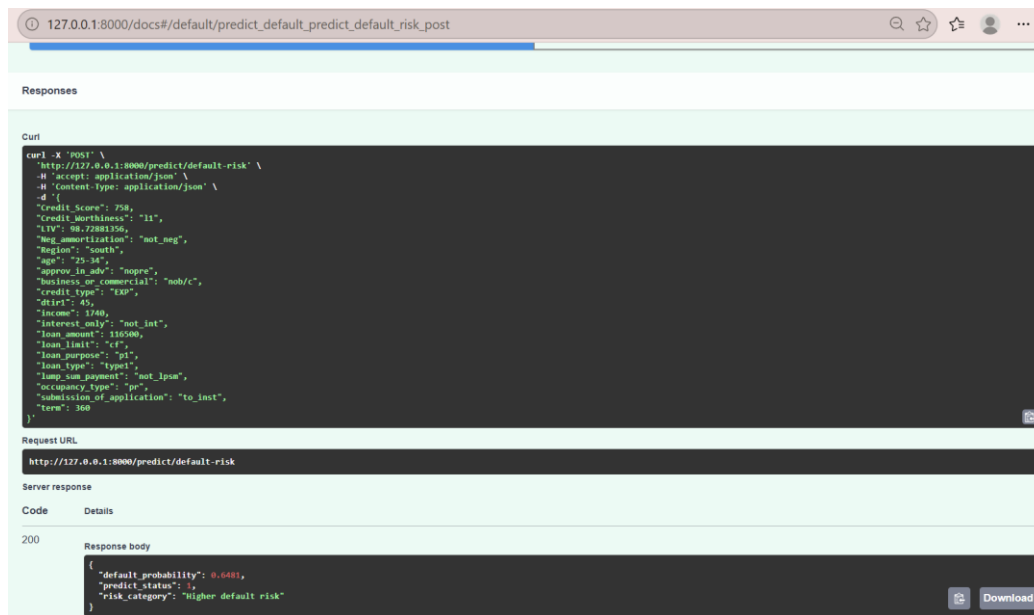
(.venv) PS C:\Users\katie\Capstone_analysis> █
```



Here, a correctly formatted API request is tested. A 200 code displayed in the terminal output, along with in the URL, with a prediction on the probability of default, the predicted loan default status being 1, and the classification of it as a high risk in default:

```
(.venv) PS C:\Users\katie\Capstone_analysis> $loanApplication = @{
>>   loan_amount = 116500
>>   term = 360
>>   income = 1740
>>   Credit_Score = 758
>>   LTV = 98.72881356
>>   dtir1 = 45
>>   credit_type = "EXP"
>>   loan_type = "type1"
>>   loan_purpose = "p1"
>>   loan_limit = "cf"
>>   approv_in_adv = "nopre"
>>   Credit_Worthiness = "l1"
>>   business_or_commercial = "nob/c"
>>   Neg_ammortization = "not_neg"
>>   interest_only = "not_int"
>>   lump_sum_payment = "not_lpsm"
>>   age = "25-34"
>>   submission_of_application = "to_inst"
>>   occupancy_type = "pr"
>>   Region = "south"
>> }
>>
>> $jsonBody = $loanApplication | ConvertTo-Json
(.venv) PS C:\Users\katie\Capstone_analysis> Invoke-RestMethod `
>> -Method Post `
>> -Uri "http://127.0.0.1:8000/predict/default-risk" `
>> -ContentType "application/json" `
>> -Body $jsonBody

default_probability predict_status risk_category
-----
0.6481 1 Higher default risk
```



Finally, an incorrectly formatted request is shown below, with the Amount variable being misspelled. A 422 code is provided both in the terminal output and in the URL, indicating that the request cannot be retrieved.

```
default_probability predict_status risk_cate$invalidApplication = $loanApplication.Clone()
>> -----
>> $invalidApplication.Remove("loan amount")Fault risk
>> $invalidApplication["loan_amunt"] = 116500
>>
>> $invalidJson = $invalidApplication | ConvertTo-Json
(.venv) PS C:\Users\katie\Capstone_analysis> try {
>>   Invoke-RestMethod `
>>     -Method Post `
>>     -Uri "http://127.0.0.1:8000/predict/default-risk" `
>>     -ContentType "application/json" `
>>     -Body $invalidJson
>> }
>> catch {
>>   Write-Host "Status code:" $_.Exception.Response.StatusCode.value__
>>   Write-Host "Response:" $_.ErrorDetails.Message
>> }
Status code: 422
Response: {"detail":[{"type":"missing","loc":["body","loan_amount"],"msg":"Field required","input":{"Neg_ammortization":"not_neg","age":"25-34","income":1740,"dtir1":45,"interest_only":"not_int","loan_amunt":116500,"LTV":98.72881356,"submission_of_application":"to_inst","Region":"south","term":360,"credit_type":"EXP","loan_limit":"cf","loan_purpose":"p1","occupancy_type":"pr","loan_type":"type1","approv_in_adv":"nopre","Credit_Score":758,"business_or_commercial":"nob/c","lump_sum_payment":"not_lpsm","Credit_Worthiness":"l1"}},{"type":"extra_forbidden","loc":["body","loan_amunt"],"msg":"Extra inputs are not permitted","input":116500}]}
```

```
curl -X 'POST' \
  http://127.0.0.1:8000/predict/default-risk \
  -H 'accept: application/json' \
  -H 'content-type: application/json' \
  -d '{
    "Credit_Score": 750,
    "Credit_Worthiness": "11",
    "LTV": 96.72881356,
    "Reg_amezization": "not_neg",
    "Region": "south",
    "age": 25.34,
    "approv_in_adv": "negre",
    "business_or_commercial": "nob/c",
    "credit_type": "EAO",
    "dtrpl": 45,
    "income": 1740,
    "interest_only": "not_list",
    "loan_amnt": 116500,
    "loan_initl": "cf",
    "loan_purpose": "pl",
    "loan_type": "type1",
    "long_run_payment": "not_ipos",
    "occupancy_type": "pr",
    "submission_of_application": "to_list",
    "term": 360
  }'
```

Request URL

http://127.0.0.1:8000/predict/default-risk

Server response

Code	Details
422	Error: Unprocessable Content

Response body

```
{
  "detail": [
    {
      "type": "missing",
      "loc": [
        "body",
        "loan_amount"
      ],
      "msg": "field required",
      "input": {
        "Credit_Score": 750,
        "Credit_Worthiness": "11",

```

An advantage of using FastAPI for deployment is that the trained model can be used for real-time predictions. A disadvantage, however, is that the deployed local prototype is not yet a full production system with monitoring, security, and scaling features.

Data Summary and Implications

The purpose of this analysis was to hopefully provide some answers to the following research questions: Which characteristics of borrowers, loans, and credit are statistically associated with loan default status, and can an optimized Random Forest classification model accurately predict the risk of defaulting on a loan? This is often important for financial companies and bank stakeholders, as they want to ensure they make the best decisions when choosing qualified borrowers for loans, ultimately avoiding missed opportunities for financial gains and potential revenue loss. Hence, they may want to understand the underlying associations between these borrowers and credit factors, and the likelihood of a loan default, to make accurate predictions and ensure they can make strategic decisions about whether to accept loan applications from borrowers based on those results.

Inferential statistical testing was performed on the selected variables to examine their statistical significance for the target variable, Status. A total of 20 distinct hypothesis tests were performed without an adjustment for multiple comparisons. Based on the results of the Chi-Square Independence Test for the selected categorical variables, all of the following variables

acquired very low p-values that were below the set significance threshold level of 0.05: credit_type, Neg_ammortization, submission_of_application, lump_sum_payment, loan_type, business_or_commercial, loan_limit, Region, age, loan_purpose, approv_in_adv, Credit_Worthiness, occupancy_type, and interest_only. Because the p-values for all these categorical variables were below the alpha threshold of 0.05, the null hypothesis is rejected, and it can therefore be concluded that these variables are statistically significant. For the selected numeric variables, Welch's T-Test and Mann-Whitney U test were performed based on whether the variables were approximately symmetric or not in distribution. The results of both tests indicate that the following variables had very low p-values below 0.05: income, LTV, loan_amount, dtir1, and term. Because the p-values for these numeric variables are also below the 0.05 threshold, the null hypothesis is rejected, and these variables are statistically significant. Interestingly, Credit_Score did not demonstrate a statistically significant univariate difference between default and non-default groups. Therefore, the null hypothesis cannot be rejected, and it can be concluded that Credit_Score did not show a statistically significant univariate mean difference between the default and non-default groups. However, this may not necessarily indicate that credit scores do not have predictive value when evaluated alongside other variables in the Random Forest classifier. Furthermore, the strength of the statistically significant associations varied with the effect sizes and p-values, as their effect sizes indicated that the associations' strengths were not equal. For example, credit type displayed the strongest association with loan default status, with a Cramer's V value of 0.592. The next-largest effect sizes were seen for the following variables: lump-sum payment, negative amortization, and application submission method, with values of approximately 0.188, 0.156, and 0.121, respectively. The rest of the categorical variables yielded smaller values, suggesting they contained weaker associations with loan default status despite their p-values showing statistical significance.

Based on the results of the initial Random Forest model, the accuracy score was approximately 0.893. The AUC-ROC score was approximately 0.88. For the default class, the initial model had a high precision score of 0.937, a recall score of 0.607, and an F1 score of 0.737. Based on the confusion matrix for the initial model, the model correctly classified 16,581 borrowers as not having defaulted and 3,338 as having defaulted on their loans. The confusion matrix also shows that the model misclassified 224 non-defaulted borrowers as having defaulted and 2,158 defaulted borrowers as not having actually defaulted.

Regarding the optimized Random Forest model, the accuracy score was approximately 0.889. The AUC-ROC score was approximately 0.892. For the default class, the optimized model yielded precision, recall, and F1 score metrics of 0.84, 0.68, and 0.752, respectively. Based on the confusion matrix, the optimized model correctly classified 16,091 borrowers as not having defaulted and 3,744 as having defaulted on their loans. The confusion matrix also demonstrates that the model misclassified 714 non-defaulted borrowers as having defaulted and 1,752 defaulted borrowers as not having actually defaulted.

A dataframe was created as a comparison table to demonstrate the performance metrics of the initial Random Forest Model in comparison to the optimized Random Forest Model:

Comparison Table of Initial vs. Optimized Model

```
model_compare = pd.DataFrame([
    'Model': [
        'Initial Random Forest',
        'Optimized Random Forest'],
    'Accuracy': [
        accuracy_score(y_test, y_test_pred_init),
        accuracy_score(y_test, y_test_pred_opt)],
    'Precision': [
        precision_score(y_test, y_test_pred_init),
        precision_score(y_test, y_test_pred_opt)],
    'Recall': [
        recall_score(y_test, y_test_pred_init),
        recall_score(y_test, y_test_pred_opt)],
    'F1-score': [
        f1_score(y_test, y_test_pred_init),
        f1_score(y_test, y_test_pred_opt)],
    'ROC-AUC': [
        roc_auc_score(y_test, y_test_proba_init),
        roc_auc_score(y_test, y_test_proba_opt)]])

model_compare.round(3)
```

	Model	Accuracy	Precision	Recall	F1-score	ROC-AUC
0	Initial Random Forest	0.893	0.937	0.607	0.737	0.886
1	Optimized Random Forest	0.889	0.840	0.681	0.752	0.892

Although the optimized model yielded slightly lower accuracy and precision scores than the initial model, it achieved higher scores of the recall, F1-score, and ROC-AUC metrics. The higher recall score signifies that the optimized model was able to identify a higher proportion of borrowers who have defaulted on their loans. Furthermore, the optimized model reduced false-negative predictions from 2,158 to 1,752, further supporting the observation of increased performance. Yet, the number of false-positive predictions increased from 224 to 714. When inspecting the F1-score, the optimized model achieved a slightly higher score than the initial model (0.752 vs 0.737), signifying that it was able to obtain a better balance between precision and recall. The precision metric, however, is lower in the optimized model (0.84) than in the initial model (0.937). This finding indicates that the optimized model had a higher number of false positives. Regarding the AUC-ROC metric, the scores of the initial and optimized models are very similar: 0.892 for the optimized model and 0.886 for the initial model. Lastly, the accuracy metric for the optimized model is 0.889, indicating moderately strong performance but slightly lower than the initial model (0.893).

Feature importance was also examined to identify which features or variables contributed most to whether a borrower is likely to default on a loan. The top five variables, or features, with the highest importance are as follows, starting with the strongest: credit type EQUI, loan-to-value ratio, debt-to-income ratio, income, and loan amount. This suggests that these features contributed more strongly to the model's predictions and may call for further review. It is important, however, to be cautious: these factors should not be quickly interpreted as independent causal factors or used solely for approval or denial standards.

The optimized Random Forest model and its preprocessing components were then successfully packaged and served through a local FastAPI application. The application was able to make a default-risk classification and model-estimated default probability for correctly formatted requests and ensured that incorrectly formatted requests would be rejected based on its validation rules. All four API unit tests successfully passed, supporting the practicality of integrating the model into a predictive application. However, this application and deployment process represents only a local prototype; therefore, further work is needed to ensure that fairness, scalability, security, and continuous monitoring protocols are established.

Overall, these results provide some clarification and answers to our focused research question. The following characteristics of borrowers, loans, and credit are statistically associated with the loan default status: credit type, negative amortization, submissions of applications, lump sum payments, loan type, business or commercial classification of loan, loan limit, region, age, loan purpose, approvals in advance, creditworthiness classification, occupancy type, interest-only loan status, income, loan-to-value ratio, debt-to-income ratio, loan amount, and term. Finally, an optimized Random Forest classification model demonstrated a moderately strong performance in predicting the risk of loan default. This is based on the accuracy metric of 0.889 and the confusion matrix, which shows that the optimized model yielded false positives. Overall, the optimized model can be an effective tool for financial stakeholders to closely examine and take appropriate measures and decisions regarding certain identified borrower characteristics when new loan applications are presented, in collaboration with financial experts.

A few limitations of this analysis are that the data do not include additional external factors that may be used in real-life lending decisions. For example, other current loans that borrowers might be on, rent or mortgage amounts, and other external factors might provide a better insight into their associations with default likelihood. Another important limitation to consider is that the EQUI credit type shows a 100% observed default rate in the heatmap visualization. This may indicate an unusually strong pattern that, while it may be a valid feature of the data, there may be other underlying reasons for this high value. Therefore, it is recommended that the model be externally validated and that the credit type be more closely examined before it is used to support financial decision-making. Another limitation is that the model may need further refinement and continued learning, as the FastAPI prototype is not the same as a production-level deployment for real-time predictions that might require ongoing monitoring and scalability. It can also continue to be trained as more data is fed to it to address fairness and bias.

A recommended course of action for this analysis is for stakeholders and financial institutions to use this model more as a decision-making tool, in combination with the expertise of financial personnel, rather than as the sole basis for approving or denying loan applications. Stakeholders can further use the statistically significant characteristics and features of greater importance to guide them as they assess default risk levels, thereby improving the consistency of this process.

A proposed future study involves analyzing fairness and bias to examine whether predictions might differ across demographic or borrower groups. This can be an important analysis to ensure that fairness and bias are addressed and that compliance is maintained before the model is used in real-world settings. Another future study could use more recent lending data that includes additional characteristics and continue to evaluate the model's performance over time. Then, the next step here would be to build a production-level deployment pipeline that is scalable and monitorable.

References

- Anand, R. (2023). Limitations of 'Pandas.' In *Medium*. <https://anand-rohan.medium.com/limitations-of-pandas-9f5f4e11e712>
- Anandgupta. (2024). `handle_unknown='ignore'`. In *Medium*. <https://medium.com/@anandgupta20588/handle-unknown-ignore-4b937a82b5e4>
- ColumnTransformer*. (n.d.). scikit-learn. Retrieved July 31, 2026, from <https://scikit-learn.org/stable/modules/generated/sklearn.compose.ColumnTransformer.html>
- Feature importances with a forest of trees*. (n.d.). scikit-learn. Retrieved July 31, 2026, from https://scikit-learn.org/stable/auto_examples/ensemble/plot_forest_importances.html
- FICO score types: Why multiple versions matter for you | myFICO*. (n.d.). Your FICO Score, from FICO | myFICO. <https://www.myfico.com/credit-education/credit-scores/fico-score-versions>
- Frost, J. (2023). Mann-Whitney U Test Explained. In *Statistics By Jim*. <https://statisticsbyjim.com/hypothesis-testing/mann-whitney-u-test/>
- GeeksforGeeks. (2025). Pandas – Filling NaN in Categorical data. In *GeeksforGeeks*. <https://www.geeksforgeeks.org/python/pandas-filling-nan-in-categorical-data/>
- GeeksforGeeks. (2025). What are the Advantages and Disadvantages of Random Forest? In *GeeksforGeeks*. <https://www.geeksforgeeks.org/machine-learning/what-are-the-advantages-and-disadvantages-of-random-forest/>
- Issparse — SciPy v1.18.0 manual*. (n.d.). Numpy and Scipy Documentation — Numpy and Scipy documentation. <https://docs.scipy.org/doc/scipy/reference/generated/scipy.sparse.issparse.html>
- Kim H. Y. (2019). Statistical notes for clinical researchers: the independent samples *t*-test. *Restorative dentistry & endodontics*, 44(3), e26. <https://doi.org/10.5395/rde.2019.44.e26>
- McClenaghan, E. (2024). Mann-Whitney U Test: Assumptions and Example. In *Informatics from Technology Networks*. <https://www.technologynetworks.com/informatics/articles/mann-whitney-u-test-assumptions-and-example-363425>
- McHugh M. L. (2013). The chi-square test of independence. *Biochemia medica*, 23(2), 143–149. <https://doi.org/10.11613/bm.2013.018>
- Part 6: Random Forest Hyperparameters & Tuning Strategies. (2025). In *Baekho Lab*. <https://baekholab.com/2025/03/22/part-6-random-forest-hyperparameters-tuning-strategies/>

Pipelines and composite estimators. (n.d.). scikit-learn. Retrieved July 31, 2026, from <https://scikit-learn.org/stable/modules/compose.html>

Ransaka Ravihara. (2022, March 19). *Sparse Matrices— What, When, and Why*. Medium; Analytics Vidhya. <https://ransakaravihara.medium.com/sparse-matrices-what-when-and-why-b2271af1fd68>

SimpleImputer. (n.d.). scikit-learn. Retrieved July 31, 2026, from <https://scikit-learn.org/stable/modules/generated/sklearn.impute.SimpleImputer.html>

Western Governors University. (2026). *Loan default dataset* [Data set]. WGU Data Science Graduate Capstone course